Design and Prototyping of an Open Air Gondola Carriage System

MECE 3390U Mechantronics

Dr. Ryan Finn

April 8th, 2026

Group: 7 (Thursday)

Due: April 12, 2026

GROUP MEMBERS			
#	Name	ID	Signature
1	William Nagassar	100893559	W.N
2	Ibrahim Razzaq	100874094	I.R
3	Layth Atyani	100826610	L.A
4	Matthew Diguer	100864031	M.D.

Abstract

This project presents the design and prototyping of an autonomous open-air gondola carriage system capable of traversing a suspended rope track. The system integrates mechanical, electrical, and control components using an Arduino Due microcontroller to achieve fully autonomous operation. A dual-motor mechanism was implemented to provide effective traction and eliminate slipping, while maintaining stable motion along the rope.

The mechanical design focuses on maintaining a low center of mass and ensuring consistent contact with the rope through a gear based clamping system. This improves stability during both ascent and descent while supporting the required passenger load. The control strategy implements an encoder based position tracking system combined with a pre-set control algorithm to regulate motion, implement required stopping intervals, and reverse direction at the ends of the track without user intervention.

Experimental testing was conducted at multiple levels, covering component testing to full system operation under load. Results demonstrated that the final design is capable of completing the required traversal while maintaining stability and reliable stopping performance. Key challenges such as insufficient motor power and battery limitations were addressed through iterative improvements, including the transition to a dual motor configuration and LiPo battery system. Overall, the final prototype satisfies the primary functional requirements and demonstrates effective integration of mechanical design, sensing, and control in a mechatronic system.

Table of Contents

1.0 Introduction

The development of autonomous mechatronic systems has become increasingly important in modern engineering applications, particularly in transportation, automation, and robotics. Systems that integrate mechanical design, sensing, and control are widely used to perform tasks that require precision and minimal human intervention. One such application is rope based transportation systems where controlled motion, stability, and safety are critical design considerations.

This project focuses on the design and prototyping of an autonomous open air gondola carriage system capable of traversing a suspended rope track. The system is required to travel along the rope, perform designated stopping actions, and return to its starting position without user input. In addition to autonomous operation, the design must ensure passenger safety, maintain traction along varying rope angles, and operate within specified time and weight constraints.

To meet these requirements, a mechatronic approach was used to integrate a mechanical drive system, electronic control hardware, and a programmed control algorithm. A dual motor gear driven mechanism was implemented to provide reliable traction and prevent slipping, while an Arduino Due microcontroller was used to manage system behavior. Encoder based position tracking was used to monitor motion and enable controlled stopping and direction changes throughout operation.

The objective of this project is to design, build, and validate a functional prototype that satisfies the specified performance requirements while demonstrating effective integration of mechanical, electrical, and control subsystems. The report outlines the design process, system development, testing procedures, and final performance of the gondola system.

2.0 Requirements

The requirements for the gondola system were developed based on the project specifications and design objectives. These requirements define the expected functionality, physical characteristics, and limitations of the system to ensure successful operation. To provide clarity and organization, the requirements are categorized into functional requirements, physical requirements, constraints, and assumptions. Each requirement is assigned an alphanumeric label to allow for easy reference throughout the design process.

Functional Requirements:

- F1: The system shall automatically traverse the rope track from starting to end point without user intervention
- F2: The system shall reverse direction and return to the starting point after reaching the end of the track
- F3: The system shall complete a full round trip within 120 seconds
- F4; The system shall perform two intermediate stops along the rope, each lasting 3 seconds
- F5: The system shall stop at the top of the track for 7 seconds before returning
- F6: The system shall maintain continuous motion without slipping during both ascent and descent
- F7: The system shall operate fully autonomously without external control inputs during operation
- F8: The system shall safely transport passengers without any loss or instability during motion

Assumptions:

- A1: The rope surface provides sufficient friction for the traction mechanism
- A2: The rope remains properly tensioned throughout operation
- A3: The battery provides sufficient power for the duration of operation

Physical Requirements:

- P1: The system shall be designed to support a total load of up to 0.454 kg (including passengers and components)
- P2: The system shall maintain a stable structure with a low center of mass to prevent tipping or oscillation
- P3: The system shall include a drive mechanism capable of generating sufficient traction force to climb the rope
- P4: The system shall include a mechanical structure that maintains consistent contact with the rope
- P5: The system shall be constructed using materials suitable for manufacturing (3D printed PLA components)
- P6: The system shall fit within the geometric constraints of the provided rope track setup

Constraints:

- C1: The system must use only one Arduino microcontroller (Arduino Due)
- C2: The system must be constructed using the provided mechatronics kit components, with limited additional parts
- C3: The system must be fully autonomous, remote control operation is not permitted
- C4: The system must complete the task in a single attempt during demonstrations
- C5: The system must not exceed the maximum weight limit of 0.454 kg
- C6: The system must operate under varying rope angles without failure

Requirement Prioritization:

The requirements were further categorized based on their importance to system functionality. Necessities represent critical requirements that must be satisfied for successful operation, while luxuries represent desirable features that enhance performance but are not strictly required.

- Necessities: F1, F2, F3, F4, F5, F6, F7, F8, P1, P2, C1–C6
- Luxuries: P3, P4, P5

The necessity requirements are directly tied to the core functionality and safety of the system such as autonomous operation, completion of the full traversal, and maintaining stability without slipping. In contrast, the luxury requirements relate to performance enhancements such as improved traction mechanisms and manufacturability considerations, which can improve system efficiency but are not strictly required for successful operation.

3.0 Background Search and design planning

The design of a rope climbing gondola system is closely related to existing research in rope-climbing robots and cable based transportation systems. These systems are used in applications such as cable inspection, maintenance, and material transport, where reliable movement along a rope or cable is required. A key challenge in such systems is achieving sufficient traction while maintaining stability and control during both ascent and descent.

3.1 Rope Climbing and Traction Mechanisms

Various rope based climbing systems have been developed in robotics. A common approach involves wheel based traction systems, where motor driven wheels press against the rope to push the rope backwards and therefore move forward in the direction. Research has shown that these systems provide a simple structure, stable climbing speed, and really good adaptability and traction to different rope conditions [1]. One of the main challenges was to develop a design that is stable and be locked on to the rope without slipping. A key reference that directly informed the design of our rope climbing gondola was the work by Ratanghayra, Hayat, and Saha (2017), *"Design and Analysis of Spring-Based Rope Climbing Robot"* [2]. This research presents a rope climbing robot that uses DC motor-driven wheels for climbing, with a spring-loaded assembly responsible for passively clamping the robot onto the rope.

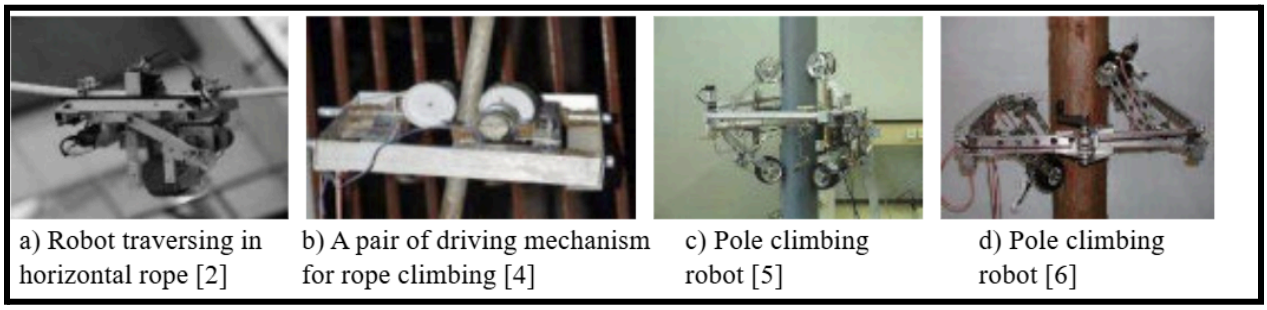


Figure 1: Different climbing Mechanisms[2]

Based on this research a gear driven mechanism was implemented where the gondola motion is driven by rotating components.This approach is very similar in principle to wheel based climbing the only difference is instead of wheels we implemented gears that are in contact with each other

and this also avoids the wheel to be slipped or comes downwards. For extra grip to the rope the ridges in between the wheel was implemented.

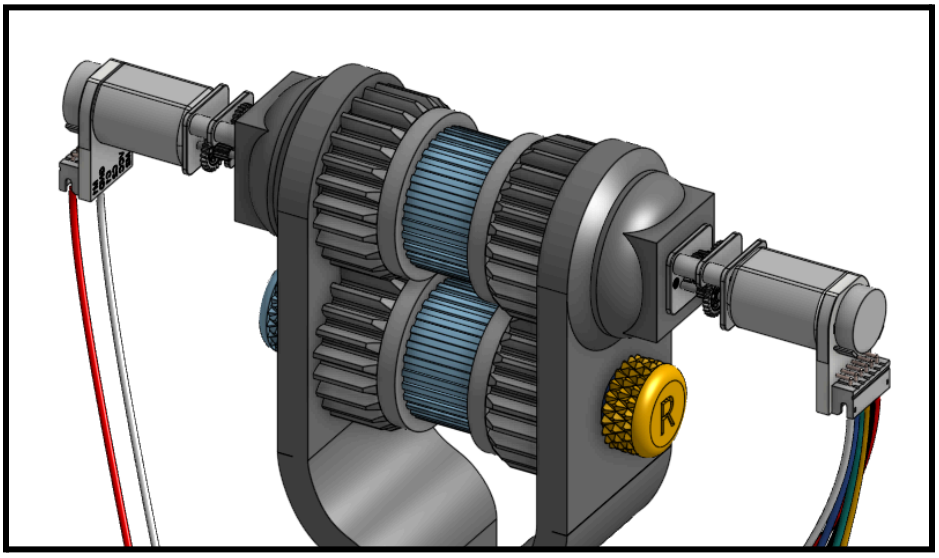


Figure 2: Gear driven mechanism

3.2 Structural Design and Load handling

The structural design for the rope climbing system must ensure stability while carrying a load, many rope climbing robots are designed with a separate drive and support mechanisms to maintain balance well and distribute weight evenly [3]. A key design consideration was maintaining a low center of mass, which was achieved by placing heavier components such as the battery and passengers in the lower section of the gondola.

This reduces the likelihood of tipping or oscillation while traveling along the rope, particularly during acceleration and deceleration. A custom CAD-designed chassis was developed to support both the drive mechanism and passenger load. The design ensures proper alignment of the rope within the system while maintaining structural rigidity. This is important for meeting the project requirement of safely transporting passengers without loss of stability.

3.3 Control Strategies for Rope-climbing Systems

Control Strategies of climbing systems generally involve either sensor based control for precise localization and positioning, or simpler control approach that rely on predefined motion without feedback. Advanced climbing robots typically use sensors such as camera, LIDAR to achieve accurate positioning and autonomous operation [4]. However these systems introduce additional complexity and require calibration and integration.

In this project the gondola motion was then decided to be programmed to accelerate, decelerate at specified intervals. This approach eliminates the need for sensors and simplifies the system design, the advantage of this approach includes reduced system complexity, lower cost, and fewer potential points of failure.

3.4 Summary of Findings

Based on the background research, several key design insights were identified:

- Wheel and gear-based traction systems are effective for rope climbing due to their simplicity and reliability.
- Maintaining strong contact with the rope is essential to prevent slipping.
- A simpler open-loop strategy was selected in order to reduce the complexity of the system, a simpler alternative to a sensor based system.

These findings directly informed the design decisions for the gondola system, leading to a solution that balances simplicity, functionality, and reliability while meeting the project requirements.

4.0 Project Management

The project schedule was developed using a gantt chart to organize tasks, track progress, and ensure timely completion of the gondola system. The project was divided into key phases, including initial planning, design development, prototyping, testing and final reporting.

The project began with brainstorming sessions and initial meetings, where roles were assigned and preliminary design concepts were developed. It also included research on programming, hardware requirements to ensure that design decisions were informed by system constraints. Following the planning phase, the team focused on detailed design tasks such as wheel design, structural stand development, and programming of the control system. Once the design components were finalized, the system was assembled and the code and components were tested in preparation for the intermediate demonstration.

The intermediate demonstration went successful but there was some refinement that needed to be made, such as the 9V battery wasn't strong enough to carry the whole thing, quickly and it was quite slow. A lipo battery was then introduced to help make it more efficient and travel up the rope quickly.

After the intermediate demonstration, the project progressed into refinement and final development. This included finalizing the code, manufacturing and printing final components, and conducting the full system assembly and testing.

Potential setbacks in the schedule included component 3-D printing, assembly alignment, and debugging of the control system. To address these risks, overlapping tasks and additional time were allocated for testing and refinement. This ensures that delays in one area don't end up impacting the whole project.

Gantt Chart

	Start Week	End Week	January				February				March					April				
			4-10	11-17	18-24	25-31	1-7	8-14	15-21	22-28	1-7	8-14	15-21	22-28	29-31	1-4	5-11	12-18	19-25	26-30
1st in person meeting (Thursday jan 29): - brainstorming and preliminary designs, distributing tasks and roles	Jan 29	Jan 29																		
Begin research on programming January 29th	Jan 29	Feb 3																		
Begin battery and hardware research January 29th	Jan 29	Feb 3																		
1st online meeting (monday february 2nd): Post Meeting Notes: - went over potential chassis designs - identified need to come up with clamping mechanism to keep wheels on rope - identified requirements such as battery size, torque requirements from motor	Feb 2	Feb 2																		
2nd in person meeting (tuesday february 3rd) Post Meeting Notes: - identified components in mechatronics kit - came up with more detailed designs for travel mechanisms and stabilization of the gondola chassis - brainstormed different programming for the motor i.e. with sensors, pre-programmed distance, etc	Feb 3	Feb 3																		
Work on wheel design February 3rd	Feb 3	Feb 24																		
Work on stand design February 3rd	Feb 3	Feb 24																		
Work on programming February 3rd	Feb 3	Feb 24																		
Assemble and test gondola design	Feb 20	Feb 24																		
Intermediate Demonstration Day	Feb 26	Feb 26																		
Work on group presentation	Feb 26	April 2																		
Finalize code	Feb 26	April 9																		
Print final gondola parts	Feb 26	April 9																		
Assemble and test final gondola design	Feb 26	April 9																		
Work on group report	Feb 26	April 10																		

Presentation Day April 2nd	April 2	April 2																	
Final Demo Day	April 9	April 9																	

5.0 Brainstorming

The following concepts were generated during the initial brainstorming phase. The goal was to explore a wide variety of mechanical, control, and material solutions which range from highly complex gear systems to simplified autonomous logic.

5.1 Drive & Actuation Mechanisms

- Dual-Motor Drive: Utilizes two independent N20 motors to provide balanced torque on both sides of the mechanism. This concept aims to maximize the climbing power and prevent lateral rotation which may occur with the single motor design.

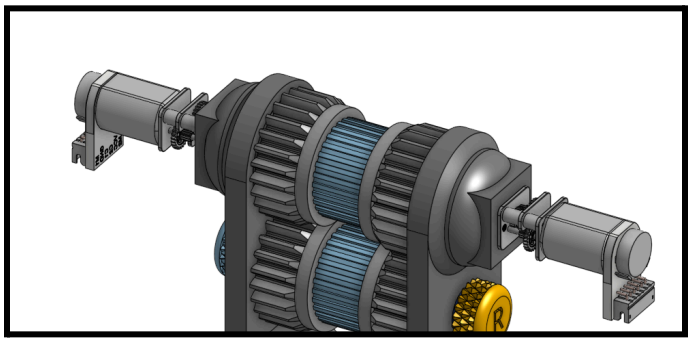


Figure 3: Conceptual CAD Model of Proposed Dual-Motor Mechanism

- Single-Motor Drive: A minimalist approach which focuses on weight reduction and wire cleanliness. With this design, a single motor drives the shaft which also reduces battery draw.

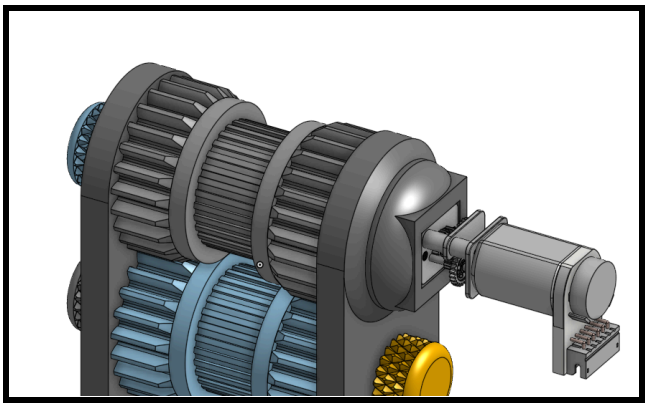


Figure 4: Conceptual CAD Model of Proposed Single-Motor Drive

- 8-Gear High Torque System: A complex gear train designed to lower the center of gravity and provide mechanical advantage, ensuring the mechanism properly extrudes the rope without rotating around it, which was a significant problem during early testing phases when there was no weight being carried by the mechanism.

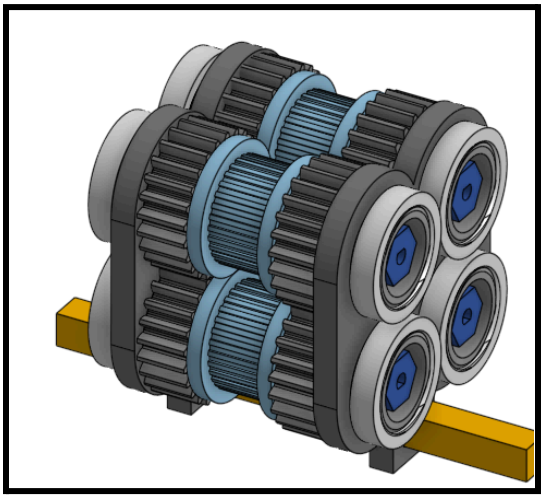


Figure 5: Conceptual CAD Model of Proposed 8-Gear System

- Extruder-Style Gear System: Inspired from 3D printer extruder technology, this approach uses two interlocking gears to pinch and pull the rope through the mechanism.

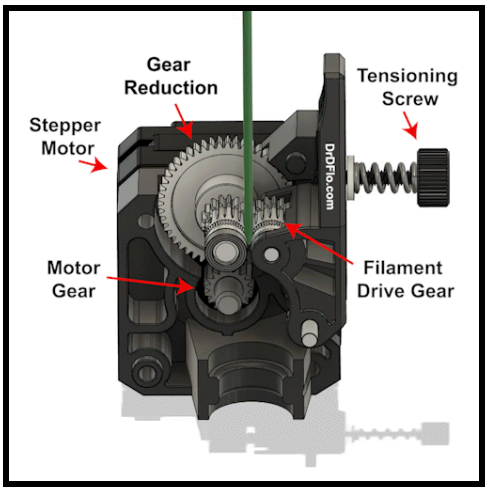


Figure 6: Annotated Cross Section of 3D Printer Extruder [5]

- Pulley-Clamped Drive: Instead of gears and rigidity to pinch the rope, this approach uses a system of pulleys located between the two drive wheels to provide clamping force on the rope.

Figure 7: Example of a Pulley-Driven Mechanism [6]

- Gravity-Fed Top Wheels: A simple mechanism where there are only top wheels above the rope, which rely on the weight of the carriage to maintain frictional forces to move.

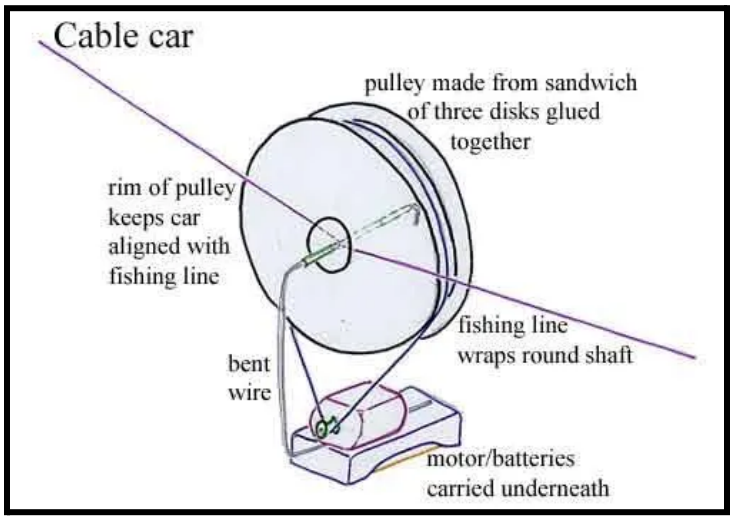


Figure 8: Conceptual Model of Gravity-Fed Top Wheels [7]

5.2 Stability & Rope Guidance

- Spring-Dampened Chassis: This approach incorporates springs which act as dampeners alongside a hinge joint to absorb vibrations and back-and-forth oscillations which will help to protect the passengers.

Figure 9: Example of Spring Dampener [8]

- Rope Guide: A rigid circular guide designed to keep the rope centered within the gear path to prevent the rope from getting pinched between the gears.

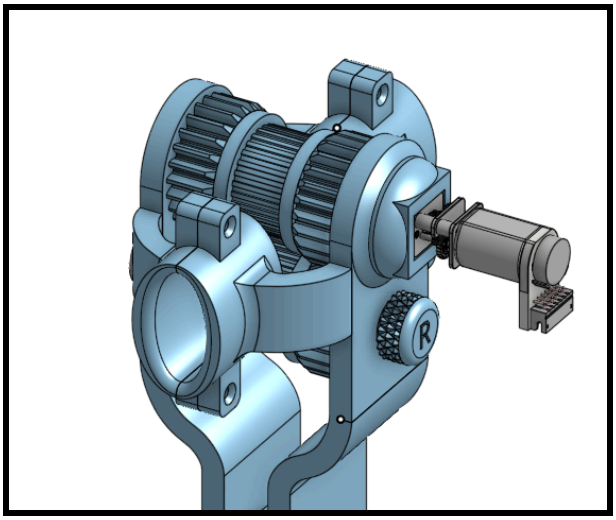


Figure 10: Conceptual CAD Model of Proposed Rope Guide

- Integrated Hinge Joint: A pivoting chassis design that allows the carriage to adjust its angle and stay perpendicular to ground.

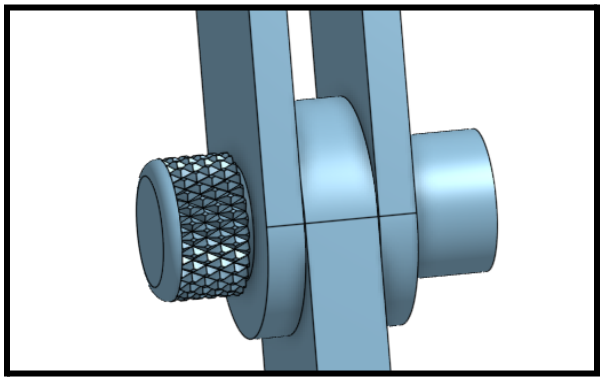


Figure 11: Conceptual CAD Model of Proposed Hinge Joint

5.3 Control Systems & Logic

- Step-Based Precision Control: This approach uses the internal step-count of the motors to calculate exact distance traveled, allowing for autonomous stops at specific lengths of the rope.

- Sensor-Based Feedback Loop: Integrates IR sensors to detect the black signal obstacles on the rope, serving as a primary or backup safety mechanism.

- Manual Remote Control: A user-operated system via a wireless transceiver, allowing for real-time adjustments and manual override of the ascent.

- Timer-Based Ascent: This approach allows the motors to run for a predetermined duration to reach the estimated target distance.

5.4 Grip & Material Exploration

- Adhesive-Enhanced Traction: This approach uses high-friction adhesives such as hot glue to increase the grip of the inner pinching wheels.

- Multi-Material TPU Wheels: This approach mixes a PLA inner wheel with TPU teeth for maximum grip.

Subsystem	Option 1	Option 2	Option 3	Option 4	Option 5	Option 6
Drive & Actuation	Dual-Motor Drive	Single-Motor Drive	8-Gear High Torque	Extruder-Style Gears	Pulley-Clamped Drive	Gravity-Fed Top Wheels
Stability & Guidance	Spring-Dampened Chassis	Rope Guide	Integrated Hinge Joint	N/A	N/A	N/A
Control & Logic	Step-Based Precision	Sensor-Based Feedback	Manual Remote Control	Timer-Based Ascent	N/A	N/A
Grip & Materials	Adhesive-Enhanced	Multi-Material TPU	N/A	N/A	N/A	N/A

Table 1: Tabulated Summary of Subsystem Concepts

6.0 Preliminary Concept Generation: Sketching Ideas

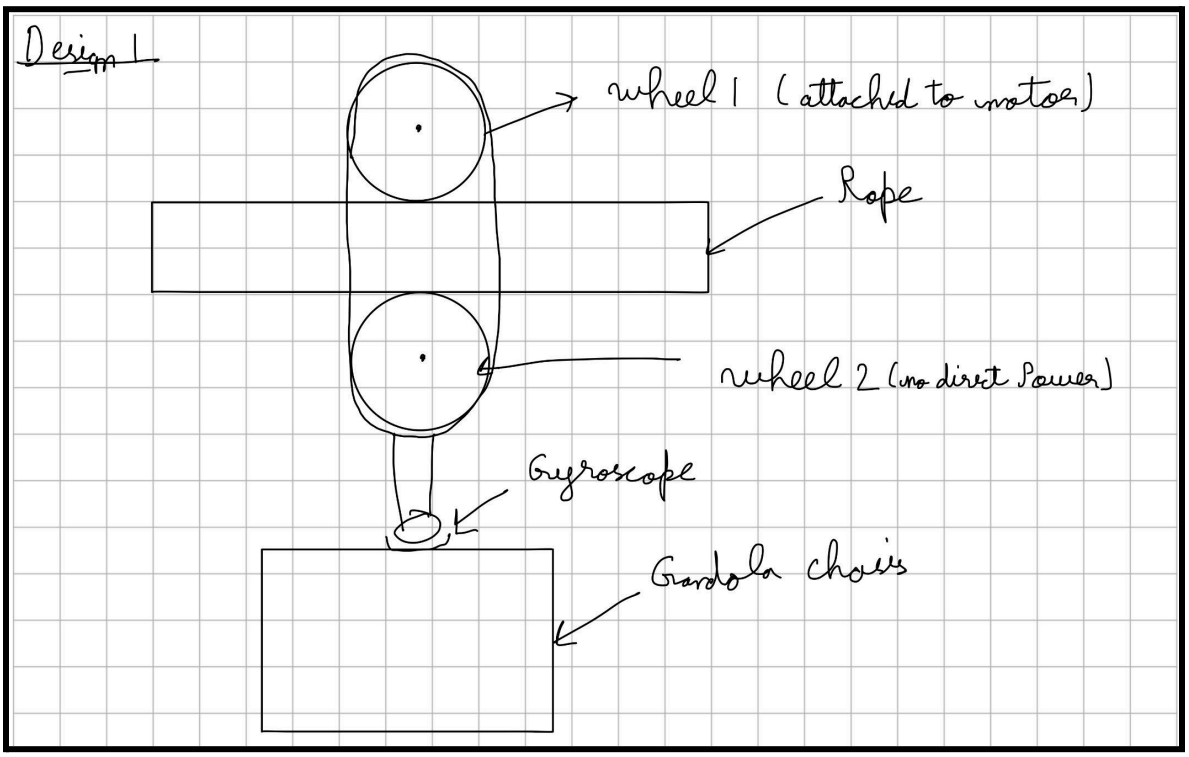


Figure 12: First design concept sketch

We started the design process after the background search, and got some inspiration from our background research. An initial design concept was developed based on using a dual-wheel compression mechanism. In this design there are two wheels which are positioned at the opposite side of the rope, applying a normal force to compress the rope between them. The top wheel is directly driven by a motor, while the bottom wheel is passive and rotates due to contact with the rope. Additionally there is also a gyroscopic joint that connects the moving mechanism to the gondola chassis, allowing the gondola to remain vertically aligned, helping to maintain stability and balance during motion along the rope. The downside for this design is that the wheels can slip, this can mainly happen when the gondola is decelerating, and when the driven top wheel starts to slow down, the contact force between the rope and the passive bottom wheel may decrease, leading to a loss of tension and reduced traction.

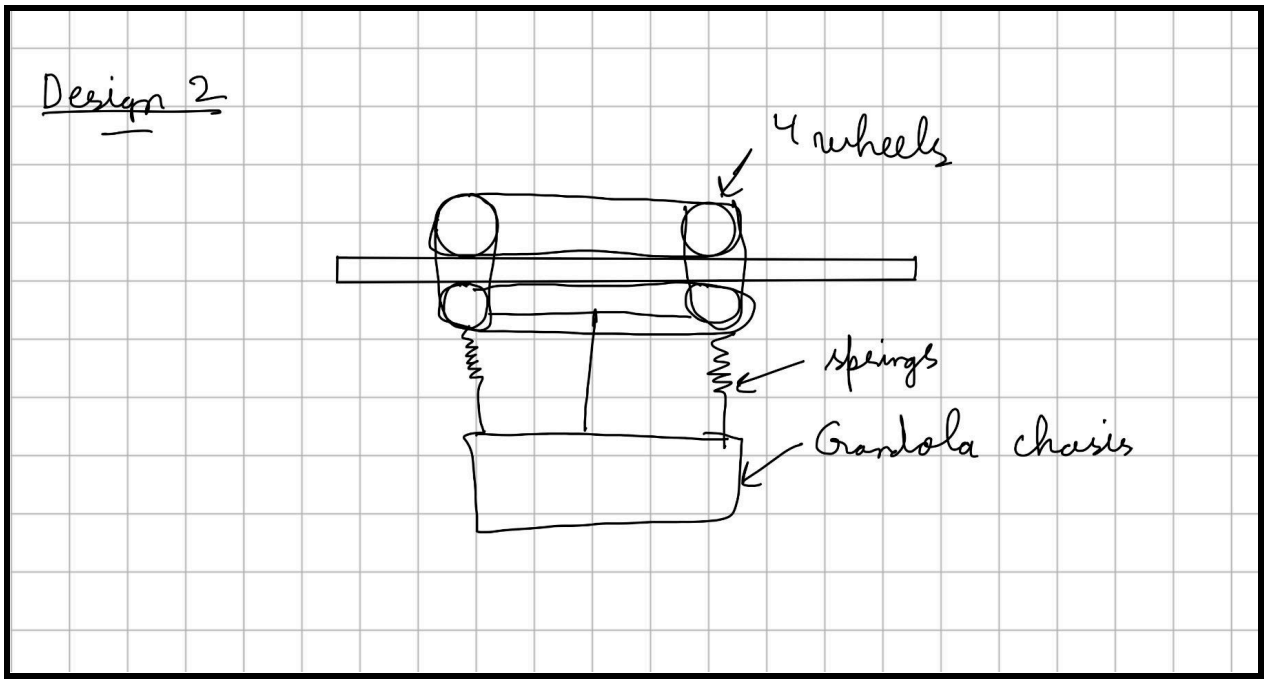


Figure 13: Second Design Concept Sketch

A second design concept was developed to improve upon the limitations of the initial dual-wheel system by increasing stability and traction. This concept utilized a four-wheel configuration, where two wheels were positioned above the rope and two below, creating a more distributed contact with the rope. The primary advantage of this design is the increased number of contact points with the rope, which improves stability and potentially enhances traction compared to the two-wheel system. However, this design introduced significant complexity. Synchronizing the motion of all four wheels required additional mechanical linkage, and maintaining uniform compression between the upper and lower wheel sets proved difficult.

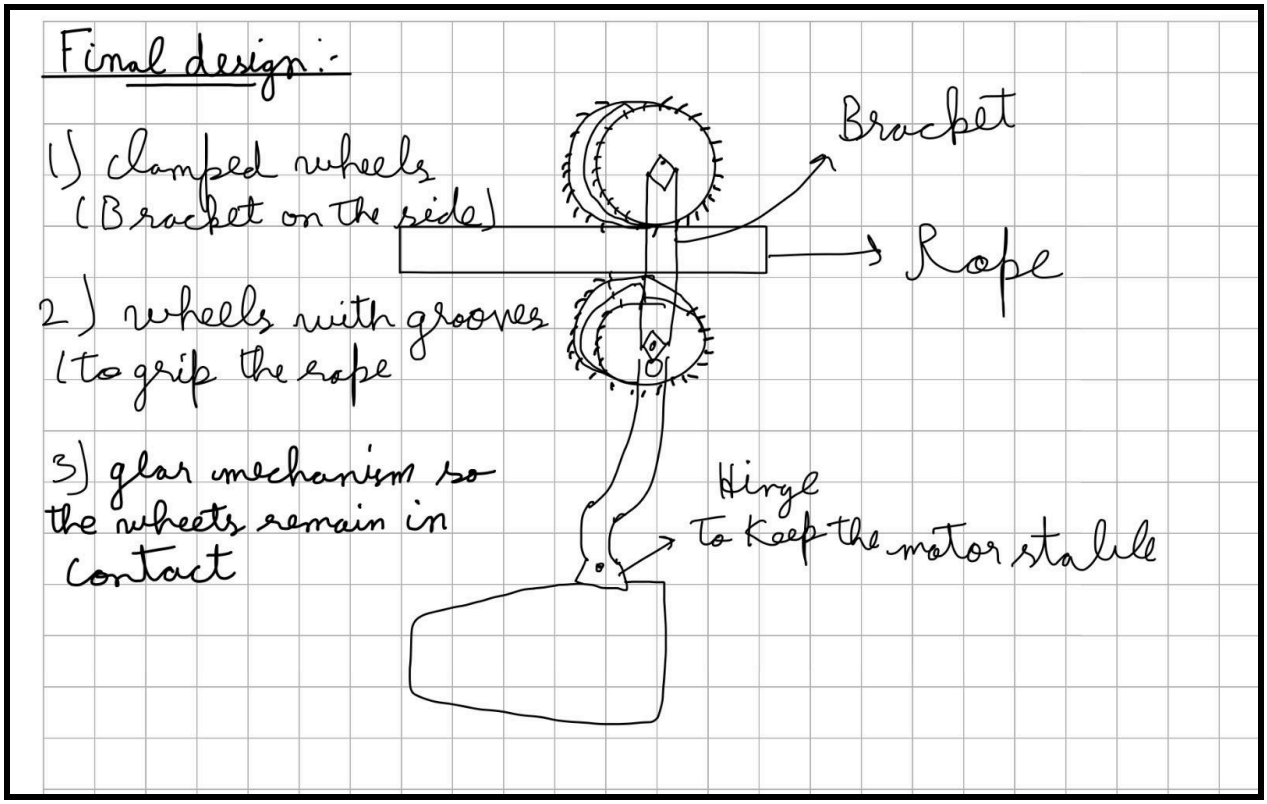


Figure 14: Final Design of gondola

The final design of our gondola system uses a two-wheel clamping mechanism to move along the rope, but with improvements over the previous concepts to reduce slipping and improve control.

Instead of using chains or rubber bands, the two wheels are held together using a rigid bracket, which keeps them consistently compressed against the rope. This ensures that the rope remains in contact with both wheels at all times, improving traction and stability.To further improve performance, the outer edges of the wheels are designed with gear-like teeth that mesh together. This means both wheels rotate together at the same speed, preventing uneven motion and reducing the chance of slipping, especially during deceleration.

In addition, the inner surfaces of the wheels include grooves (ridges) that increase friction with the rope. These grooves help the inside surface of the rope to grip more effectively, allowing the system to generate enough traction to move the gondola upward while carrying a load.

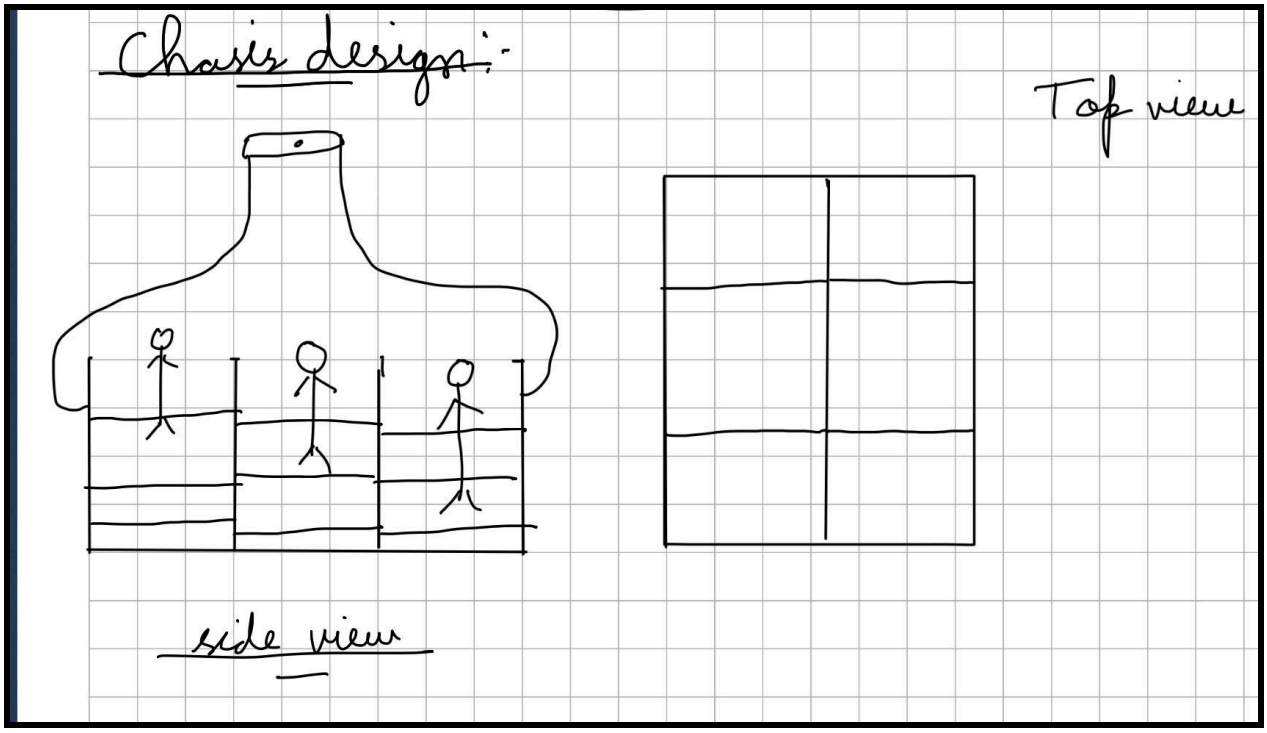


Figure 15: Final Chassis Design

The final chassis design was inspired by the shape of an egg carton, where each passenger has their own defined space. This layout allows the passengers to stand comfortably without being restricted or pressed against each other. The compartments help keep each passenger secure while still leaving enough open space so the gondola feels less enclosed and more open-air. This design also improves stability by evenly distributing the weight across the chassis, while still allowing passengers to have a clear view outside the gondola.

7.0 Engineering Specifications

In order to verify how good our design is functioning some engineering specifications were developed to quantitatively evaluate how well the design meets the project requirements. Each requirement was translated into a metric with a defined target value.

Each requirement was translated into a measurable metric with a defined target value. For example, the requirement to complete the task efficiently was represented by total travel time, with a target of less than 120 seconds. Similarly, passenger safety was evaluated by ensuring zero passenger drops during operation. These specifications were used to guide the design process and assess system performance some of the engineering specifications developed from the requirements are down below.

Requirement	**Engineering Specification**	**Target Value**
Completing Trip quickly/efficiently	Total Travel Time	≤ 150 seconds
Passenger safety	Number of passengers falling	0
Maintain Traction	Slip Occurrence	0
Required stops	Stop duration accuracy	±0.5 sec
Lightweight Design	Total System Mass	≤ 0.454 kg
Autonomous operation	Human Intervention	0
Reliable operation	Successful runs	100% Completion

Table 2: Engineering Performance Specifications and Target Values

8.0 Detailed Concept Generation: Functional Decomposition

To better understand the system's architecture, a functional decomposition was created, as shown in Figure 16. This block diagram breaks down the self-stabilizing gondola design into its core functional subsystems.

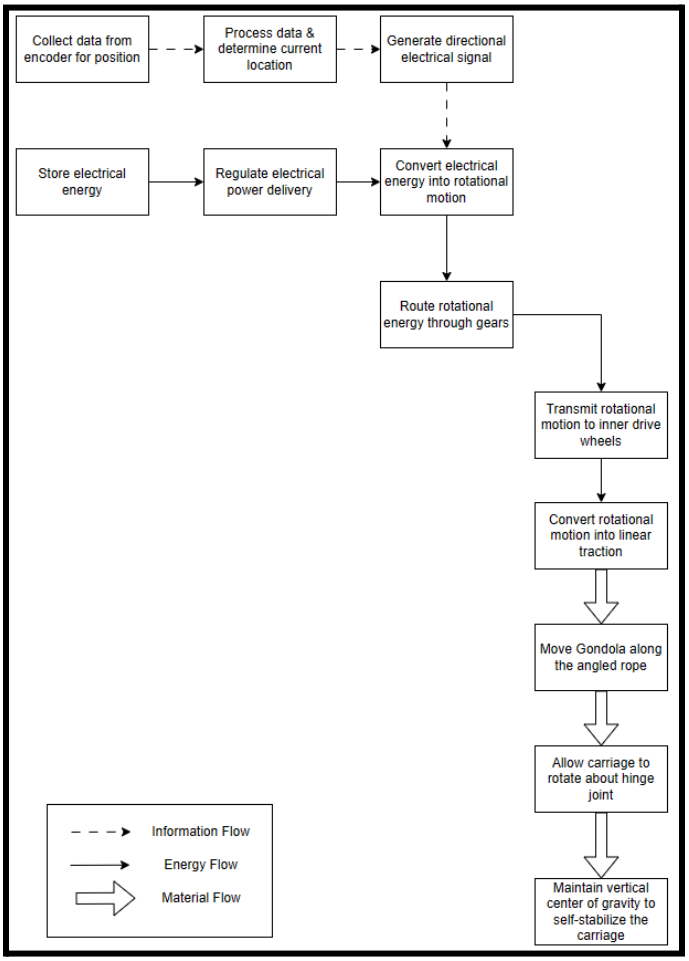


Figure 16: Functional Decomposition of Gondola System

As illustrated, the system relies on an information loop where encoder data dictates the electrical signal, which in turn regulates the flow of electrical energy to the drive wheels. Additionally, the material flow highlights the mechanical progression from linear traction to the ultimate goal of self-stabilizing the carriage via the hinge joint as it moves along the rope..

9.0 Concept Development and Screening/Selection

In concept development, 3 basic designs are proposed, and the best, based on a combined score, is selected. For the purpose of this report, the final design is labelled as “Final”, and the others “Concept 1” and “Concept 2”. Concept 1 and Concept 2 come from the designs featured in Figures 12 and 13, respectively from section: 6.0 Preliminary Concept Generation: Sketching Ideas. Additionally, for analysis purposes it is assumed that concept 1 uses a timer to determine when to stop, and concept 2 a sensor.

Requirement	**Engineering Specification**	**Target Value**	**Weight**	**Concept 1**		**Concept 2**		**Final**	
				Score	**Weighted Score**	**Score**	**Weighted Score**	**Score**	**Weighted Score**
Trip efficiency	Total Travel Time	≤ 150 seconds	4	3	**12**	4	16	5	**20**
Passenger safety	Number of passengers falling	0	5	4	**20**	5	25	5	**25**
Maintain Traction	Slip Occurrence	0	2	2	**4**	5	10	3	**6**
Stop accuracy	Stop duration accuracy	±0.5 sec	4	2	**8**	3	12	4	**16**
Mass	Total System Mass	≤ 0.454 kg	2	5	**10**	2	4	4	**8**
Autonomous operation	Human Intervention	0	3	2	**6**	4	12	3	**9**
Reliable operation	Successful runs	100% Completion	5	3	**15**	4	20	5	**25**
Total			N/A	N/A	**75**	N/A	**99**	N/A	**109**

Table 3: Decision Matrix for the Different Concept Designs

From the table above, the final design is chosen to be the most effective for the chosen purpose of this project. The first design, while light is deemed less efficient by having a set of standard wheels which the team feared could risk sliding. Concept 2 features two sets of wheels, thus reducing the probability of slipping though being much heavier due to this. The final design uses

a tight gear-based mechanism and keeps a set of wheels gripped tight over the rope, maintaining traction while being a fairly lightweight design.

10.0 Form Design & Engineering Analysis

10.1 Odometry and Kinematics

To accurately track the robot's movement, the distance traveled was calculated using the gear radius and encoder pulses. The total distance (d) is determined by dividing the total number of pulses recorded (P_total) by the pulses per revolution (P_rev), and multiplying by the circumference of the gear ($2\pi r$).

Using a gear radius of 12.5 mm:

$$d = \left(\frac{P_{total}}{P_{rev}}\right) * (2\pi r)$$

$$1000 = \left(\frac{P_{total}}{2940}\right) * (2\pi(12.5\,mm))$$

$$P_{total} = 37433$$

To stop just a small amount before the 1 meter mark, a P_total of 36500 was chosen.

10.2 Mechanical Design & Tolerances

- Motor Mounting: Initially, the N20 motors lacked proper axial constraints, relying only on a friction fit to prevent axial movement. To secure the motors and lock all degrees of freedom, M1.6 screw mounts were introduced. This ensures the motors remain rigidly attached to the bracket under load.

- Print Tolerances: Precise tolerances were critical, particularly for torque transmission. The shaft bore was oversized by 1 mm to allow a snug fit and eliminate backlash. The N20 shaft mounting hole required strict tolerances; a loose fit would cause shaft play and inefficient torque transfer. By perfectly dialing in high-performance PLA and calibrating

the printer, a CAD tolerance of 0.0 to 0.1 mm resulted in an ideal slight press fit in the post-processed parts.

- Gondola Rope Clamping: The center gap was iteratively tested to find a balance between overcoming dynamic frictional forces and maintaining sufficient clamping force to hold the gondola steady without motor load. Initial gaps of 5.6 mm, 5.5 mm, and 4.5 mm were tested. Once gear meshing issues were resolved, a final gap of 4.0 mm was found to be ideal.

- Center of Gravity (COG) Optimization: During initial unloaded testing, the gears tended to rotate around the rope rather than driving the robot forward. To lower the COG and keep the drive system oriented correctly, a testing carriage filled with copper BBs was utilized.

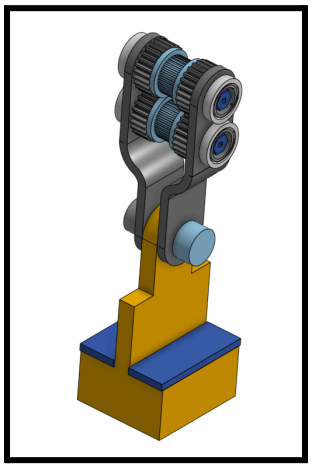


Figure 17: Gondola with Chassis to fill with Bbs

- Shaft Clearance: The top hexagonal shaft was prone to getting stuck inside the bracket's square pocket as it spun, causing the motor to stall.

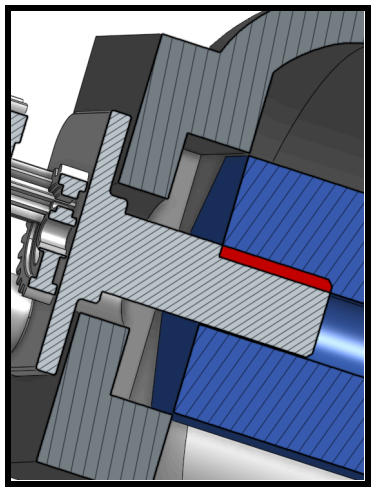


Figure 18: Shaft Pocket which causes Shaft to get Stuck

The pocket was redesigned to decrease the internal clearance area, ensuring the outer faces of the hex shaft barely clear the pocket walls without binding.

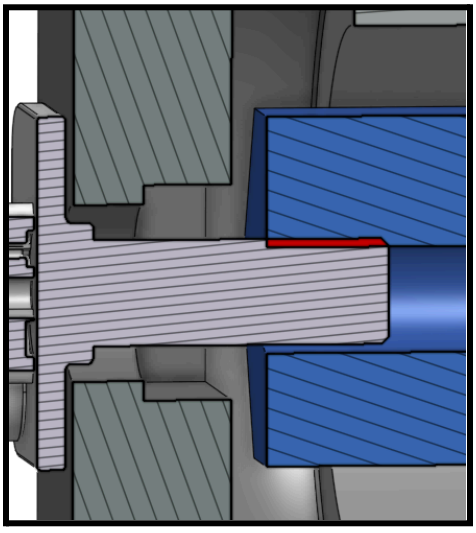


Figure 19: Redesigned Shaft Pocket

- Thread Locking Mechanism: Friction against the bracket caused the bottom shaft screw caps to repeatedly unscrew. This was partially mitigated by using opposing threads (embossed "L" for left-hand and "R" for right-hand threads), which reduced unscrewing by 50%. To fully secure the assembly, PTFE tape was applied to the threads, chosen for its ease of use and high reliability compared to superglue or locking washers.

10.3 Electrical & Power Systems

- Wiring Resistance: Initial testing revealed poor motor performance, traced back to high electrical resistance from wires spliced together with electrical tape. All connections were subsequently soldered, significantly reducing resistance and increasing motor efficiency.

- Power Source Selection: The system originally utilized a 9V battery for the motors and a separate battery bank for the Arduino. The 9V battery's high internal resistance severely limited current output, causing brownouts and poor motor performance. The dual-battery setup also weighed a combined 170 g. The system was upgraded to a single 900 mAh 2S 30C LiPo battery (7.4V), which easily supplies the high current required for peak motor performance under load. This also brought the Vin voltage within the Arduino Due's ideal 6–16V range. This switch reduced the power supply weight to just 39 g, which is a 77% weight reduction.

- Current Draw & Circuit Protection: Component current draws were estimated to determine appropriate wire sizing and fuse ratings.

Component	Average Current (mA)	Max/Stall Current (mA)
N20 Motors (x2)	340	2000
Deek-Robot L298P Motor Shield	30	30
Arduino Due	150	150
Total	520	2180

Table 4: Estimated Electrical Current Requirements of System Components

Based on the calculated maximum stall current of 2.18 A, a 3 A fuse was selected to prioritize safety. The wires connecting the battery to the board's Vin and Ground were accordingly upgraded from 1 A to 5 A ratings. The Deek-Robot L298P shield supports 2 A per channel for short bursts (1.3–1.5 A realistically). Under normal operation, the system will not stall long enough to damage the board thermally. However, if an unexpected fault causes the current to exceed 3 A, the fuse will blow, protecting the motor shield and downstream components.

11.0 Design for Manufacturing

11.1 Geometric Optimization & Overhang Management

To minimize the need for support material and improve surface finish in critical areas, geometry was optimized according to a general 45-60 overhang limit.

- Chassis Orientation: The chassis was designed from ground up to be printed vertically. This orientation aligns the largest surface areas with the Z-axis, which greatly reduces the need for supports and improves the structural integrity of the base. Arduino mounting pins would be printed vertically here and require supports, so to account for enlargement in the z-axis, the pins were undersized to 0.28mm to accommodate the 0.32mm arduino mounting holes. The hinge hole was also allowed supports, as the enlargement in this case would be good for a loose fit.

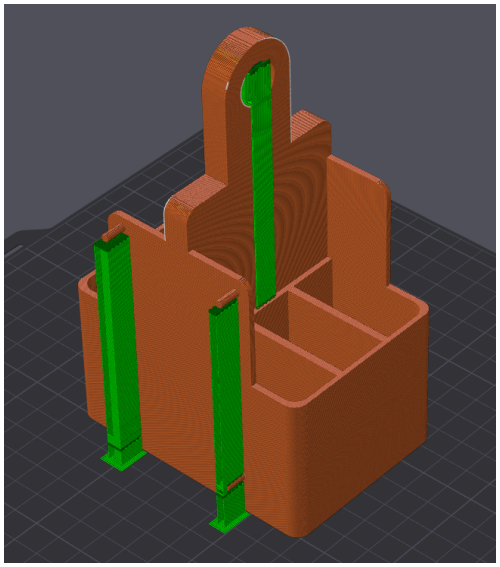


Figure 20: Chassis Print Orientation with Support Structures

- Bracket Splines: The curvature of the bracket splines were mainly limited to a 60° angle, as overhangs of this angle are generally well within the range of the Bambu P1S using PLA. An exception for this was at the beginning of the spline where

curvature was near parallel to the Z-axis, however this was only for a short distance and it was expected that the printer would be able to handle this “bridging”.

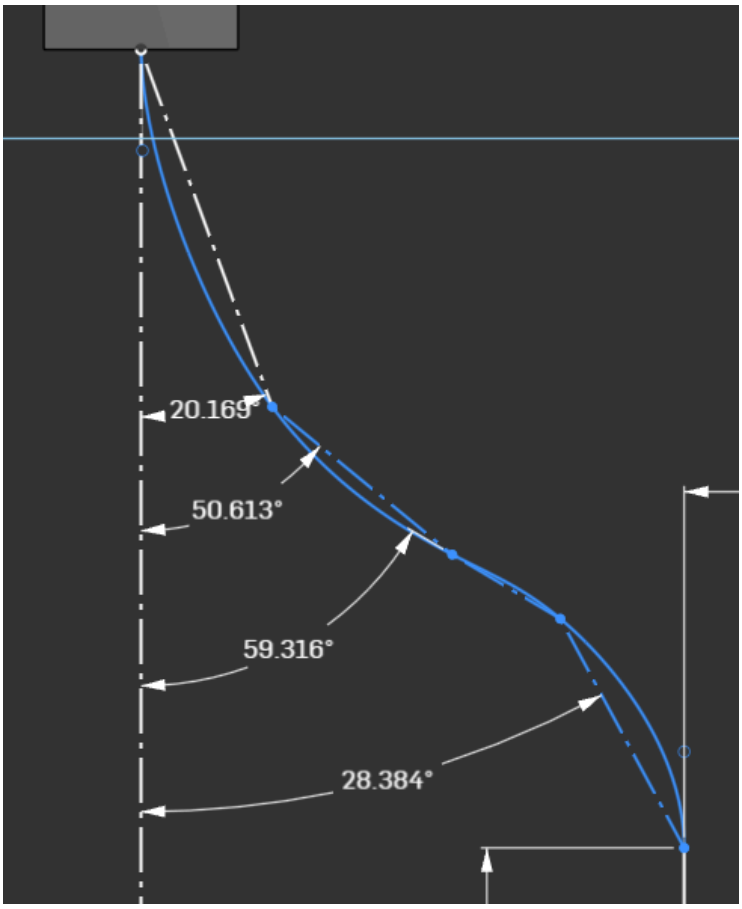


Figure 21: CAD Sketch for Bracket Spline

- Surface Texture: While the outer surface of the brackets has aesthetic artifacts from the supports, it has no impact on the movement mechanism. This was a strategic choice in order to be able to have the inner surface of the bracket, which interfaces with the gears, as smooth as possible. The ironing feature was also used for the inner surface. The ironing feature was strategically used for the interfaces of the gears and bracket. This helped to reduce friction which would put more stress on the motors and shaft. The slicing output can be seen in the Figure below, along with the post-processed bracket surface finishes.

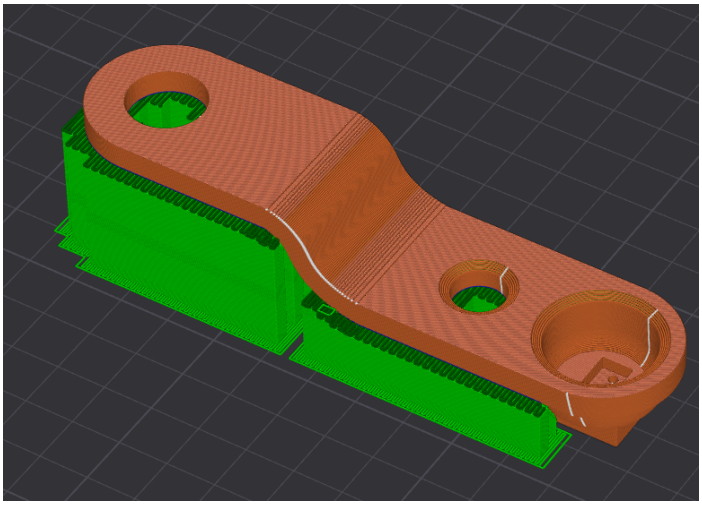


Figure 22: Bracket with Ironed Inner Surface

Figure 23: Rough Outer Surface of Bracket

Figure 24: Smooth Ironed Inner Surface of Bracket

11.2 Print Orientation & Structural Integrity

3D printed parts are inherently anisotropic, with the layer lines which are typically printed on the Z-axis weaker than the X/Y axes.

- Shaft Torsion: The drive shafts are printed vertically, meaning motor-induced torque acts as a significant shear force across the layer due to frictional forces from the rope. To lower the risk of layer separation, the shaft was printed with 100% infill to maximize layer adhesion.

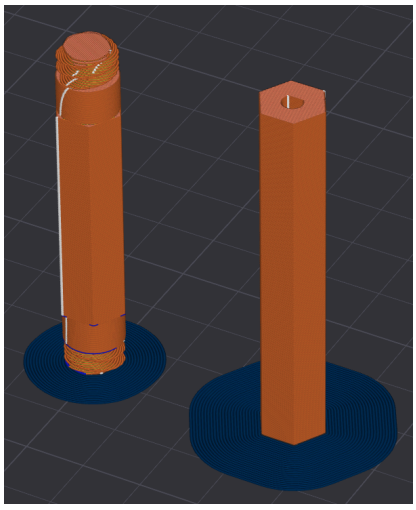


Figure 25: Sliced Shafts with Vertical Print Orientation

- Infill Strategy: Non-critical components utilized a 15% gyroid infill. This pattern was chosen because it provides nearly isotropic strength in all directions and would give these components a good general strength.

11.3 Assembly & Precision Features

- Modular Gear System: The gear system was split into three pieces, being the inner wheel and the two outer gears as seen in the Figure 26 cross section. A 60° chamfer was made on the gear after the spacer to eliminate overhangs and ensure that the part can be printed flat on the build plate for maximum tooth accuracy.

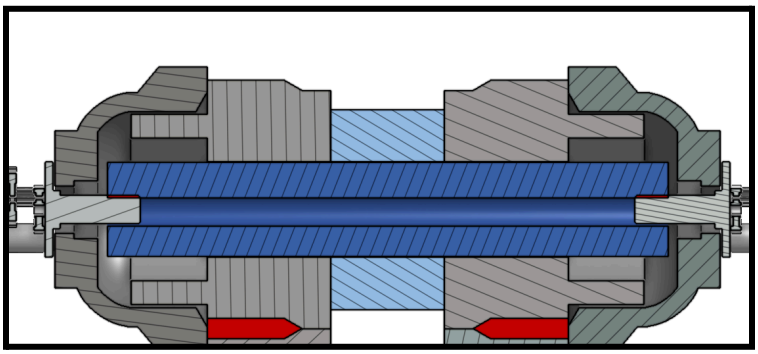


Figure 26: Cross Section View of Shaft

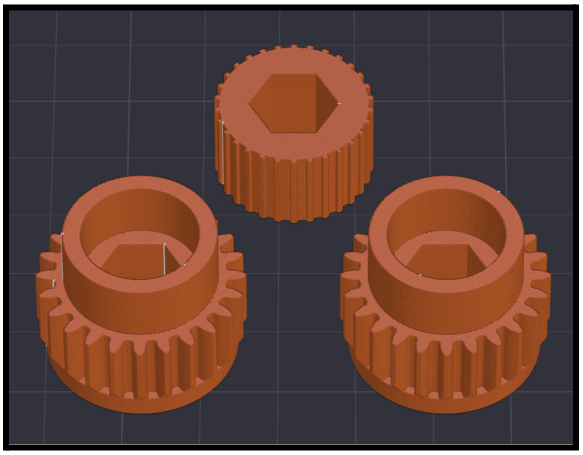


Figure 27: Sliced Shaft Components with 0 Overhangs or Supports

- Alignment Strategy: The gear system components were designed to be assembled directly on the shaft and then secured with superglue. This allowed for the shaft to be swapped out in case of the motor shaft stripping the bore and also ensured that all components were concentrically aligned.

- Threads & Ergonomics: Threaded caps and knurled textures were modeled to fit the resolution of the Bambu P1S. The overhangs on these threads and knurls do not exceed the 60° threshold, which allows for highly accurate threads and patterns

11.4 Material Selection & Post-Processing

- Material: Polylactic Acid (PLA) was selected for its superior dimensional stability and low thermal expansion coefficient. This ensures that critical component holes, such as the motor mounting holes and N20 shaft pocket, can be modelled accurately.
- Tolerance Handling: For the Arduino pin slots, supports were used because the dimensions here were not critical. If they did not fit the tolerance, then we simply use them to line up the Arduino and attach double sided sticky tape to fix it to the carriage.

12.0 Safety (Design for Safety)

Failure Mode Effects Analysis								
System / Function	Failure Mode	Failure Cause	Failure Effect	Preventive Action	Severity (S)	Occurrence Probability (O)	Detection (D)	RPN
Drive System	Gear Slipping	Insufficient friction or clamping force	Gondola fails to climb	Increased grip from ridges on gears	9	5	4	180
Motor System	Insufficient torque	Low power supply	Gondola stops moving	Dual motors, effective battery selection	8	4	3	96
Control System	Incorrect Stopping / Timing	Programming errors or encoder misread	Missed stops or incorrect motion	Pre tested algorithm and validation	6	3	3	54
Power System	Battery Depletion	Insufficient battery capacity	System stops mid operation	Switch to use of LiPo battery	7	3	2	42
Electrical System	Short Circuit or Wiring Failure	Loose Connections or Improper Wiring	System shutdown or damage	Fuse protection, secure wiring	7	2	3	42
Structure	Chassis Instability or Tipping	High Center of Mass or Uneven Load	Gondola tilts or becomes unstable	Design with low center of mass	8	3	4	96
Rope Connection	Loss of Contact with Rope	Improper Alignment or Spacing	System slips	Rigid clamping mechanism and proper alignment	9	3	5	135
Carriage	Passenger Falling	Structural Failure or Instability	Injury / Failure of Requirement	Secure and stable design	10	2	4	80

A Failure Modes and Effects Analysis (FMEA) was conducted to identify potential failure modes within the gondola system and evaluate their impact on system performance and safety. Each failure mode was assessed based on its severity (S), likelihood of occurrence (O), and ability to be detected (D). The severity, occurrence, and detection ratings were assigned based on engineering judgment, considering the impact of each failure on system safety, the likelihood of occurrence based on testing and design complexity, and the ease of identifying the failure during operation. A Risk Priority Number (RPN) was calculated for each failure mode to prioritize risk mitigation strategies which was found through the following formula: RPN = S * O * D. Higher RPN values were used to identify and prioritize critical failure modes requiring design modifications or mitigation strategies. Preventive measures were implemented into the design to minimize the likelihood and impact of critical failures.

The FMEA results indicate that the most critical risks were associated with traction and contact loss with the rope, as these failures had the highest RPN values. These risks were addressed through the implementation of gear based clamping and dual motors to improve traction and system stability. The least critical risks were found to be electrical and control system related. They had the lowest RPN due to the use of protective measures such as a fuse and pre-testing of the algorithm to ensure functionality. Overall the design incorporates appropriate measures to reduce high risk failures and ensure safe and reliable operation of the gondola system.

13.0 Test Plan, Test Results, and Product Validation

13.1 Low Level Tests

Test Description	Expected / Targeted Result	Actual Test Results
Actuator Speed (Battery Variance)	Determine optimal voltage for torque/speed balance.	9V not able to supply necessary current, needs additional battery pack to prevent shutting down. LiPo was able to give the desired performance without any issues.
Actuator Max Speed (Selected Battery)	Achieve a round trip on the rope in under 2 min using LiPo.	Round trip time of 53 s achieves the sub 2 min goal.
Interval Start/Stop/Slowdown	Precise control of deceleration to prevent jerking.	Successfully tested different deceleration ramps to combat inertia while avoiding jerking.
Gear Sizing & Inner Wheel Gap	Maximum clamping force without stalling motors.	Different iterations showed the most optimal gap is 4 mm.
Carriage Hinge Smoothness	Zero-jerk rotation during carriage adjustment.	1 mm gap between the hinge joint and hinge gives a fluid, frictionless movement.
Wheel Grip Materials/Patterns	Maximum friction coefficient on rope surface.	Hot glue did not bond to PLA and just peeled off. 1 mm by 1 mm teeth on the inner wheel give sufficient grip on the rope.
Shaft Retaining Mechanisms	Zero axial movement of shafts during operation.	Used left and right threaded caps to prevent chances of cap unscrewing and increased gap between cap and bracket to reduce friction on the cap. Additionally PTFE was wrapped on the threads to increase resistance to unscrewing. This lead to

		secure retention of the bottom shaft.
N20 Motor Mounting Torque	Screws remain secure under high reaction torque.	N20 was securely mounted still after numerous round trips on the rope.
Fuse & Switch Functionality	Reliable power cut and circuit protection.	Electrical safety verified; instant cutoff.

Table 5: Low Level Component Testing and Results

13.2 Mid Level Tests

Test Description	**Expected / Targeted Result**	**Actual Test Results**
Drill-Powered Mechanism Test	Validate rope climbing geometry without motors.	The mechanism successfully traversed rope.
Hybrid Power (9V + Battery Pack)	Offset 9V current sag during high-load ascent.	The current provided was barely enough and the mechanism was very slow.
N20 Navigation (Unattached)	Logic/Code validation for navigation commands.	The navigation system responds to logic inputs.
Dual Motor Speed Synchronization	Both motors spin at identical RPM to avoid drift.	RPM matched via PWM calibration.
LiPo No-Load System Check	Stable power distribution to all electronics.	All subsystems powered; no voltage drops.

Table 6: Mid Level Subsystem Testing and Results

13.3 High Level Tests

To systematically evaluate the system's performance and iterate on the design, three distinct hardware configurations were tested during the development process:

- **V1:** Single-motor configuration powered by a 9V battery (for motors) and a separate battery bank (for logic).
- **V1.5:** Dual-motor configuration utilizing the initial 9V battery and battery bank power setup.
- **V2:** Final dual-motor configuration powered by a high-discharge LiPo battery.

The table below outlines the high-level tests conducted on these configurations:

Test Description	Expected / Targeted Result	Actual Test Results
V1: 0° Rope Climb (Basic)	Successful ascent with movement mechanism only.	Passed; smooth movement at 0°.
V1: 30° Rope Climb (+200g)	Ability to lift extra weight at an incline.	Rope climbing is slow and the mechanism rotates laterally depending on direction of motor spin, leading to the rope getting stuck between gears.
V1: 30° Start/Stop Control	Controlled movement on incline without sliding.	Precise stops were achieved without sliding backwards
V1: 30° Start/Stop (+200g)	Maintain control under maximum load.	Precise stops were achieved without sliding backwards but instead of decelerating smoothly the mechanism stops and stalls the motor after a few deceleration steps.
V1: Intermediate Demo Setup Test	Complete V1 assembly run following intermediate demo procedure.	V1 was too slow to climb the rope and often got stuck due to the additional friction from the rope guide needed to keep the mechanism centered.
V1.5: Dual Motor 30° Climb	Increased power efficiency with dual motors.	V1.5 passed with a noticeably higher torque on the incline and is able to complete the test although slowly due to the 9V not being able to provide the necessary current.
V2: LiPo Dual Motor 30° Run	Full system validation with high-capacity power.	V2 has enough torque to maintain a steady speed throughout the whole incline, and accelerate quickly after stops. The LiPo battery provides the two motors with sufficient current.

Table 7: High Level System Testing and Results

14.0 Automation Algorithm Design

Below are the flowcharts following the code for the gondola system the team designed. For effective viewability, the flowchart is split into five section:

- Initialization
- Void Loop function
- SpeedChange function
- motorControls function
- Encoder function.

These flowcharts represent the different sections of the code. **Initialization** is where pins and variables are defined appropriately, and all else, where applicable, is initialized. The **Void Loop function** represents the main loop of the code. This is where the program cycles, determining whether motors should be running or paused, and determining the appropriate direction of the rotation of these motors. The **SpeedChange function** is called whenever the main Void Loop section decides to change the speed or direction of motors. This section gradually slows the motors down to a stop or gradually increases their speeds from a stop to the default speed. For each step of gradual speed change, the **motorControls function** is called to update the motor. Finally whenever channel A of the encoder detects movement, the **Encoder function** is called, and the motor position variable is updated accordingly.

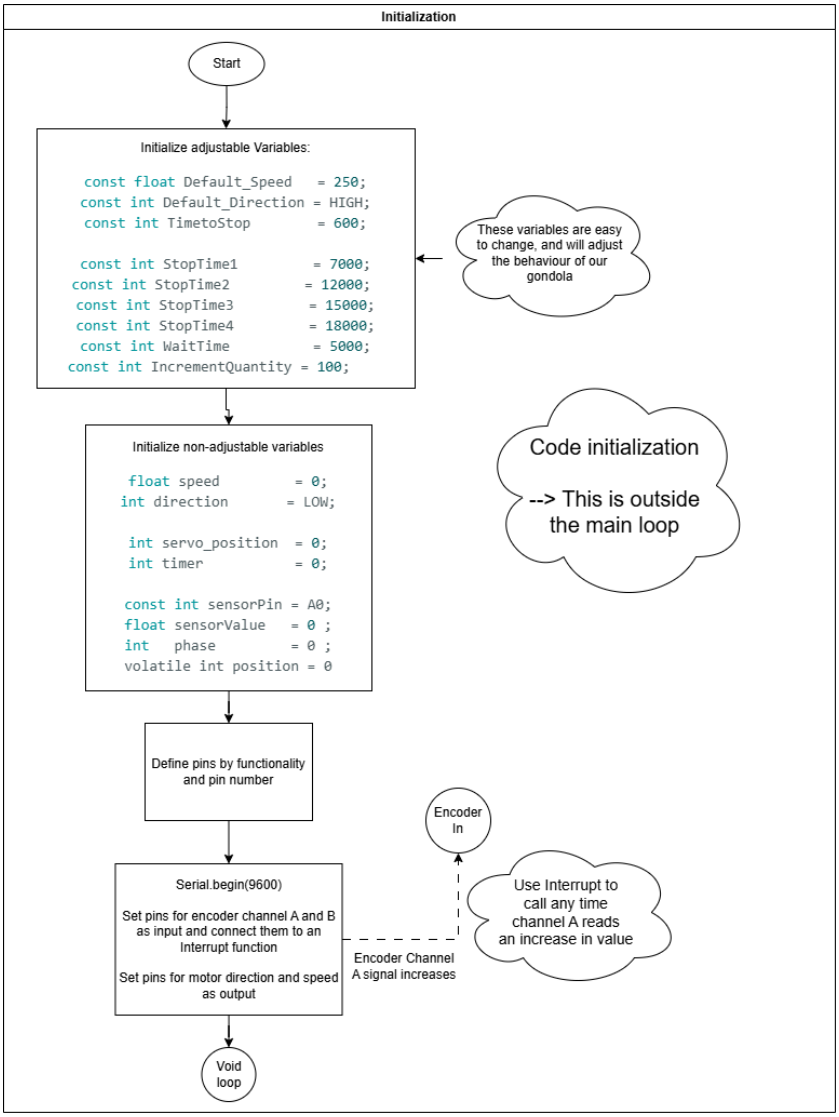


Figure 28: Initialization section of flowchart

Here the necessary steps are taken to initialize the code. Variables are defined and the correct parts of the hardware are defined as being connected to the correct pins (more on hardware setup in Electrical Hardware Setup, page 52). Additionally, the interrupt function for the encoder is defined here, specifically under void setup().

The interrupt function interrupts current code activity whenever it receives activity from the encoder's channel A, and reads the **Encoder function** to update the motor's read position. This ensures the motor's position is continuously updated as the gondola continues to move and change.

The variables in the initialization section are defined as follows:

Variable Name	Variable Definition
Default_Speed	The standard speed of travel of the motor
Default_Direction	The starting motor rotation direction
TimetoStop	Time the motor takes from beginning to stop (or start) and coming to a full stop (or returning to Default_Speed)
IncrementQuantity	Amount the motor speed changed by for each cycle during the motor speed changing process (see the **SpeedChange function**)
speed	Current motor speed
direction	Current motor direction
timer	Code timer -> counts how long (roughly) the code has been running for
sensorPin	Sensor's pin is defined as a variable for when it is read in the **Void Loop function** section of the code.
position	Represent motor's position, calculated from the **Encoder function**

Table 8: Variables in Initialization section of code

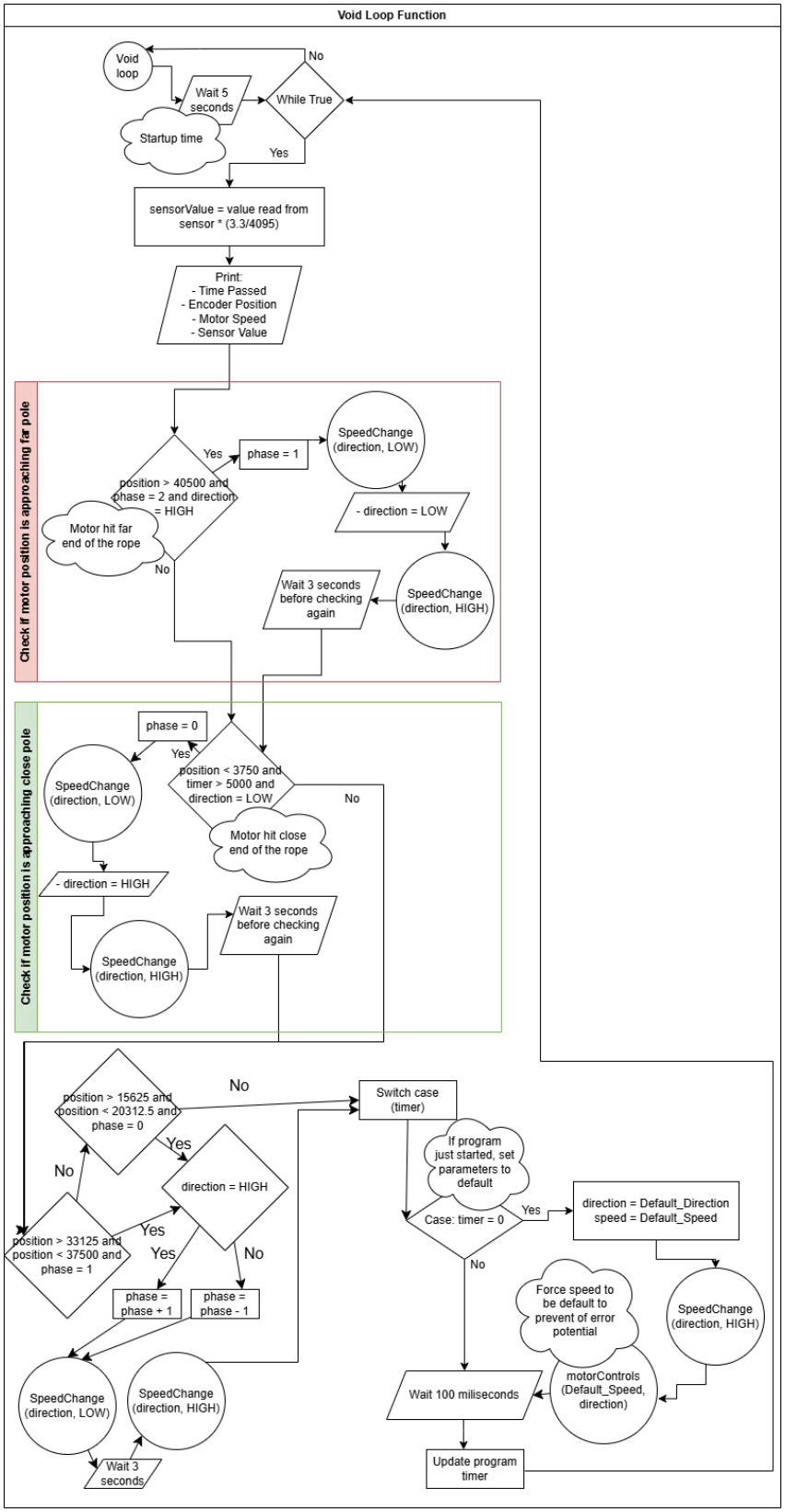


Figure 29: Void Loop function flowchart

Above is the main Void Loop section of the code. This is the main segment of code that loops continuously and which calls separate sub functions to make adjustments when appropriate. This code begins by waiting 5 seconds (to give the motor sufficient startup time, then enters a permanent While(true) loop.

In this loop the code reads from the sensor (if attached), then prints current motor data including the time variable, motor position, motor speed and sensor's last read value as a debug statement.

The code then checks (using the encoder and the measured length of the track) if it is currently close to either the far or close pole. If it detects the motor has surpassed the measured position, the motor stops calling the SpeedChange function, then changes its rotation direction and turns itself back on, again calling the SpeedChange function. The if and else-if also ensure the motor direction is moving forwards (for the far-end stop) or backwards (for the close-end stop) before triggering, ensuring the stop is not triggered more than once on accident. After the motor has been set to rotate in the opposite direction, the code changes the variable phase to 1 if the motor is moving backwards or 0 if forwards. This variable is used for stops ensuring a stop isn't triggered more than once on a single trip while the motor is in the stop range. The motor stops for 7 seconds, as required at both ends of the track. The two end-point conditional statements are represented through the red and green boxes in the flow chart, representing the far end and close end of the track, respectively.

If the code time = 0 it determines the motor has not begun moving yet and thus updates the motor speed and direction to the default speed and direction (calling the SpeedChange function).

For the intermittent stops, another else-if statement is used. It measures when the encoder falls within a specific, measured margin and if the phase variable is equal for that respective stop (0 for the first stop and 1 for the second stop). When the condition is met, the motor stops for 3 seconds before resuming motion. The stop ranges for the motors are determined via trial and error, measuring the required stop locations for both the upwards/forwards and downwards/backwards motions of the gondola on the rope with a marker, then adjusting the stop

location until it aligns with the marked spots. The phase variables ensure a stop doesn't trigger multiple times on accident while the motor is within a designated stop range. Each stop adds to or subtracts from the phase variable, depending on if the motor is moving forwards or backwards, respectively. The phase variable must equal that written in the else-if statement for the stop, thus ensuring each stop occurs once per loop.

After this the program continues allowing the motor to move in its current direction while waiting 100 milliseconds, then updates timer to reflect the time waited and finally loops.

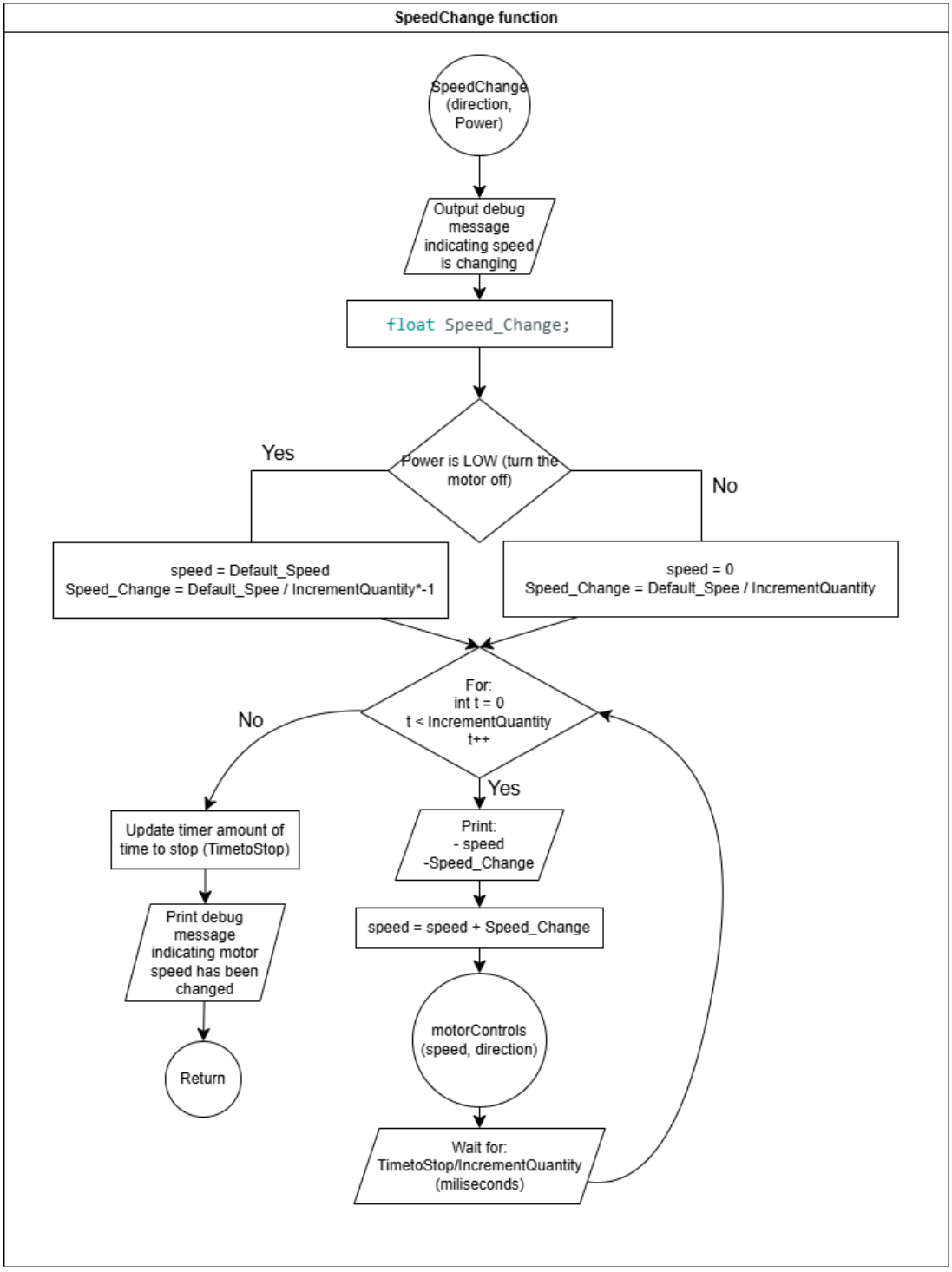


Figure 30: SpeedChange function flowchart, function used to set speeds gradually

This function focuses on controlling the speed of the motor in a gradual manner. More specifically, this function, when called, gradually increases or decreases the speed to/from the default speed. This gondola is designed to have the motor rest at either the standard movement speed or be fully stopped. When the motor needs to stop to change direction or to allow passengers to disembark at their respective stops, or when the motor needs to return to its initial speed after a stop it is important the motor gradually slows to a stop or gradual return to full speed for the comfort of the passengers as well as long-term lifespan of the gondola.

This function automatically applies a gradual stop/start with the parameters defined in table 9. Speed_Change is auto calculated to represent the amount the motor speed changes by on each incremental speed change. For each change in speed, the motorControls function is called then waits a calculated amount of time based on the parameters from table 9. After the speed is updated, the variable “timer” is updated accordingly. The motorControls function is left as a separate function for program organization, and for ease of future feature implementations, such as potential emergency stop programming.

This function receives direction and Power as inputs which determine the motor’s rotation direction as well as whether it is returning to default speed (Power = HIGH) or slowing to a stop (Power = LOW).

Variable Name	Definition	Type
Power (sent to function in calling statement)	HIGH = Power On -> Turn motor back on LOW = Power Off -> Turn motor off	bool
direction (sent with calling statement	Motor rotation direction clockwise vs counter clockwise	bool
Speed_Change	Incremental speed change amount	float

Table 9: SpeedChange function variables

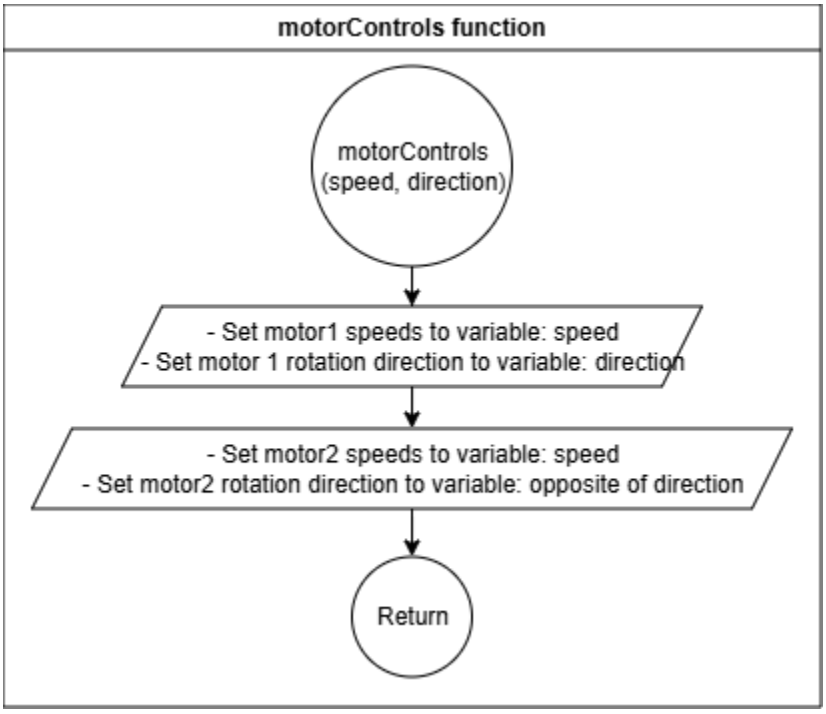


Figure 31: motorControls function flowchart

The motorControls function simply receives speed and direction input when called, and applies it to both the motors. This is placed as a separate function for organizational purposes.

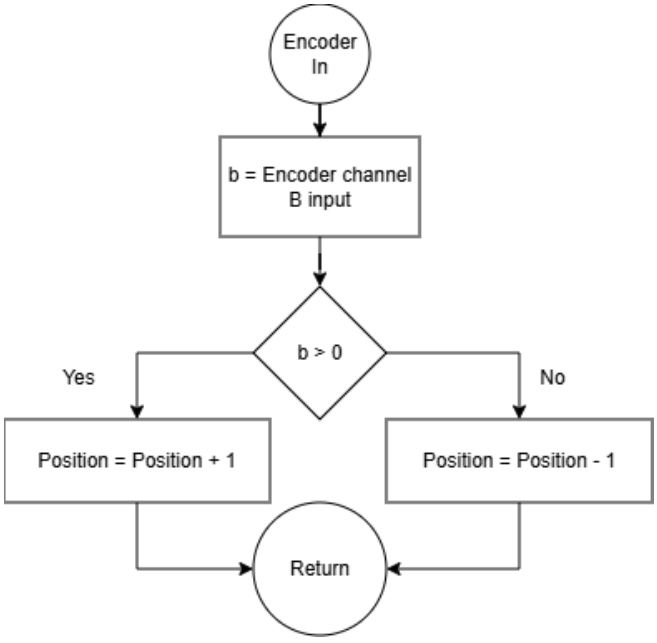


Figure 32: Encoder flowchart

When an increase in the encoder channel A is detected, this function is called. From there if channel B > 0 the program knows channel B on the encoder was already called and thus this is determined as the "forwards" direction of rotation and the position is incremented by 1. If the channel A value is called and B = 0 the encoder must be rotating in the opposite direction. This is taken as "backwards" and thus decrements the position value by 1.

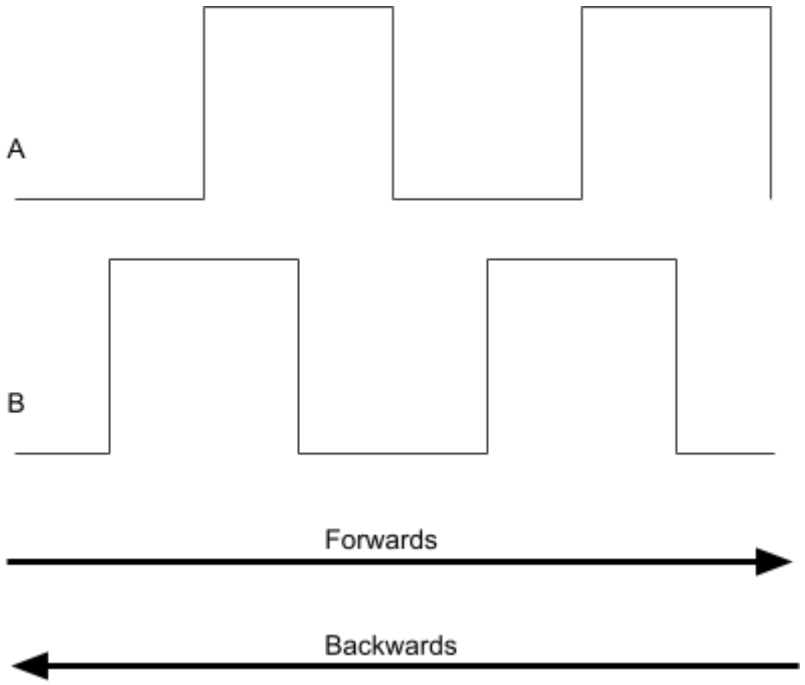


Figure 33: Encoder Pulse Diagram

Above is a representation of encoder pulses. As shown above, as the encoder is rotating (or "moving") in a forwards direction, when A is called B has already been called. If rotation "backwards" when A is called B has not yet been called.

15.0 Electrical Hardware Setup

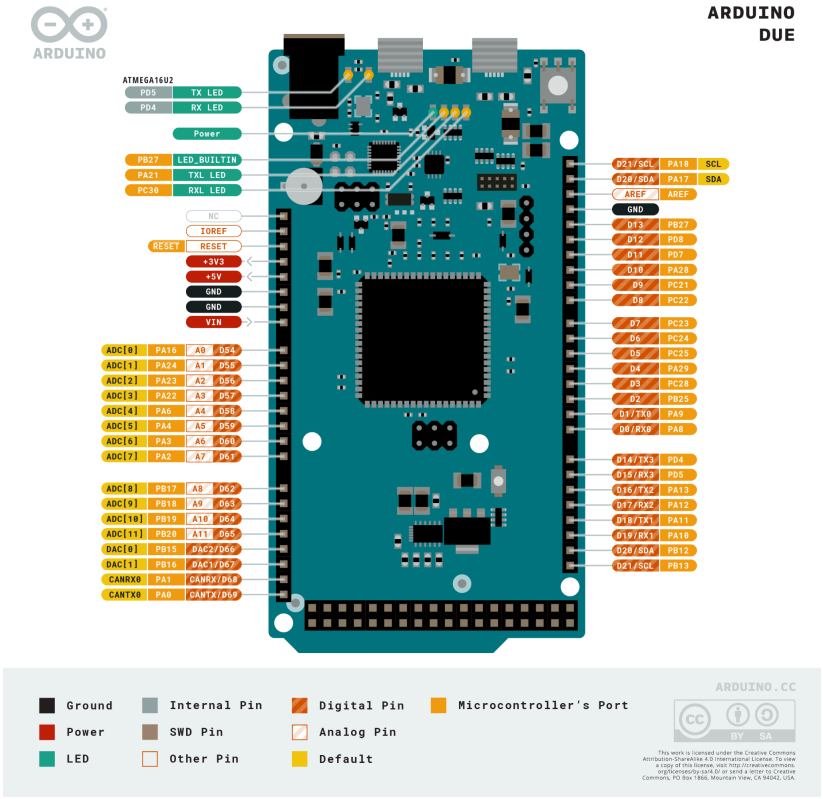


Figure: 34: Arduino due board setup. Figure taken from [9]

Above is a schematic diagram of the Arduino DUE board, taken from the official arduino website, source [9]. This figure presents the many pins across the Arduino DUE board.

Part	Quantity
Arduino Due	1
Deek-Robot Motor Shield	1
FIT0485 Geared Motor with Encoder	2
3A Fuse	1
Switch	1
LiPo Battery	1

Table 10: BOM for electronic parts of the gondola assembly

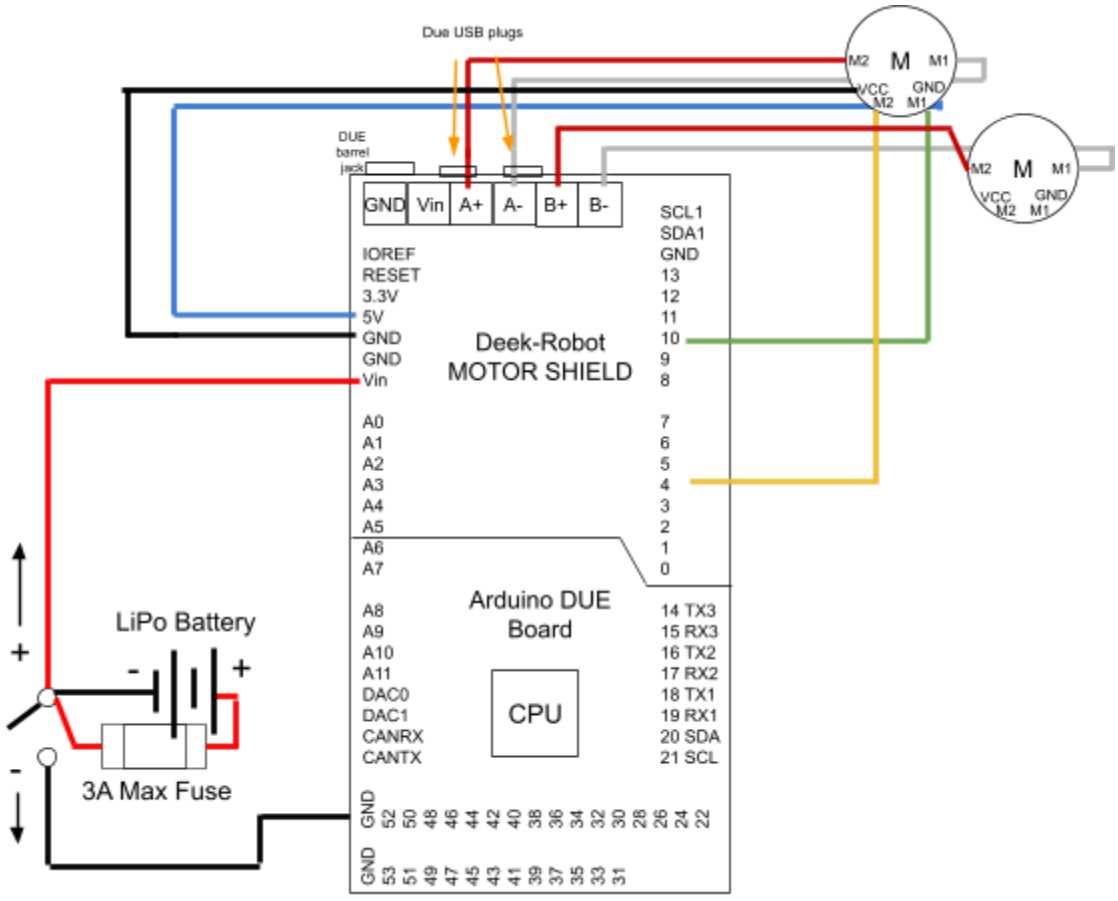


Figure 35: Schematic illustration of the electronic parts of the gondola device. Note: light grey is used to represent white wires.

As illustrated above, the final design used features a rather simple, yet nonetheless effective electronic setup. The Deek-Robot motor shield is plugged directly into the Arduino Due board. The motor shield is powered separately through the attached LiPo battery. Thus the motorshield powers the Arduino board through the LiPo battery and the two boards' connected pins. Connected to this battery are a switch and a 3 Amp fuse. This switch provides an easy method to turn on and off the device without unplugging wires from the Arduino board or motor shield. The fuse serves as a layer of protection for the motor shield and Arduino DUE board, preventing shorting and other electrical damage should anything misbehave.

Two gear motors are also plugged into the motorshield, and both gear motors have encoders built onto them, one of which is also plugged into the motor shield pins. These two gear motors are connected to the main gondola device and rotate it. Note that because of the orientation of the motors in placement, they are programmed to rotate in opposite directions. The encoder in use is plugged into the 5V pin for power, GND directly below for grounding, 10 and 4 for encoder channels A and B, respectively.

When powered, the Arduino DUE behaves as the main processing unit of the system. This board sends commands to the motor shield on top, and the motor shield adjusts the motors' powers as required. While doing so, the encoder channels return constant outputs as the motor moves. The DUE receives these as inputs and uses them to internally calculate the motor's position with the written encoder function (see figure 35). If this position is determined to be beyond a set point (indicating the board has reached one of the ends of the track) the DUE tells the motor shield to slow, then stop the motors before changing rotation direction and turning them back on. Thus the motor shield gradually stops the motors, changes rotation direction then gradually rotates again. The same process occurs for stops along the track.

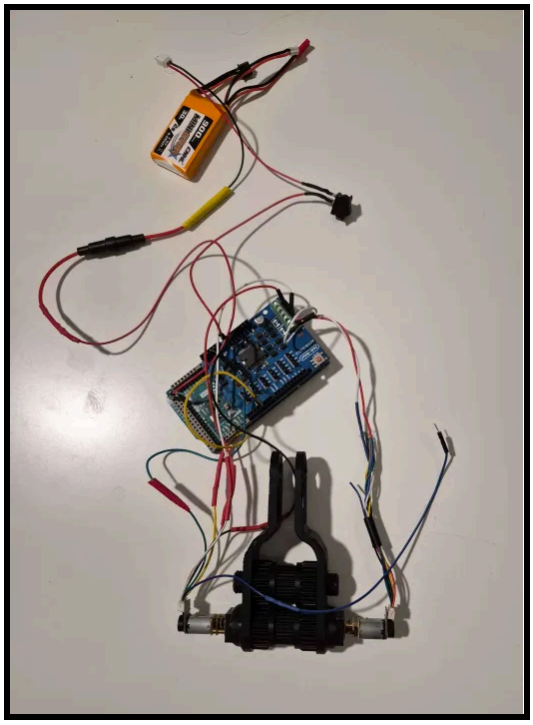


Figure 36: Photo of gondola electronic/mechatronic parts

16.0 Conclusion

The objective of this project was to design, prototype, and validate an autonomous open air gondola system capable of traversing a suspended rope track while meeting performance requirements. Based on the final system performance and testing results, the designed gondola successfully achieved all of its goals.

The final design demonstrated effective autonomous operation, this included completing a full round trip with the required time, executing intermediate and endpoint stops, and reversing direction without user intervention. The implementation of a dual motor system and gear based clamping mechanism significantly improved traction and eliminated slipping issues found in earlier designs. Additionally, the switch to a LiPo battery provided sufficient current to ensure consistent performance under load and solve power limitation issues observed during initial testing.

Experimental validation across low level, mid level, and high level testing confirmed that the system operates effectively under various conditions. As seen in the test results, the final configuration was capable of maintaining stable motion on the inclined rope sections while carrying the specified load, with smooth acceleration and deceleration through the programmed control system. The system achieved all required safety and performance requirements including maintaining passenger stability and preventing any structuring or operational failures.

In conclusion, the developed gondola system provides a functional and reliable solution, highlighting effectiveness of design, testing, and system integration in mechatronic engineering applications.

References

[1] G. Fang and J. Cheng, "Design and implementation of a wire rope climbing robot for sluices," *Machines*, vol. 10, no. 11, p. 1000, Oct. 2022, doi: 10.3390/machines10111000.

[2] A. A. Hayat, "Rope climbing robot: Design and analysis," Academia.edu. [Online]. Available: https://www.academia.edu/37497628/Rope_Climbing_Robot_Design_and_Analysis. [Accessed: Apr. 12, 2026].

[3] J. Wang, L. Yin, R. Du, L. Huang, and J. Huang, "Design and analysis of a dual-rope crawler rope-climbing robot," *Mechanical Sciences*, vol. 15, no. 1, pp. 31–45, Jan. 2024, doi: 10.5194/ms-15-31-2024.

[4] B. Tao, Z. Gong, and H. Ding, "Climbing robots for manufacturing," *National Science Review*, vol. 10, no. 5, Art. no. nwad042, Feb. 2023, doi: 10.1093/nsr/nwad042.

[5] Dr. D-Flo, "Building a 3D printer: Extruder," Drdflo.com. [Online]. Available: https://www.drdflo.com/pages/Guides/How-to-Build-a-3D-Printer/Extruder.html. [Accessed: Apr. 12, 2026].

[6] "V-belt pulley," IndiaMART. [Online]. Available: https://www.indiamart.com/proddetail/v-belt-pulley-23178263797.html?srsltid=AfmBOortf2Uq7YGTJMEZvuaCsw6spGFadZLKfiHaPh7FWTnsa0TwyJ5S. [Accessed: Apr. 12, 2026].

[7] "Cable car," Durealeyes.com. [Online]. Available: http://durealeyes.com/cablecar.html. [Accessed: Apr. 12, 2026].

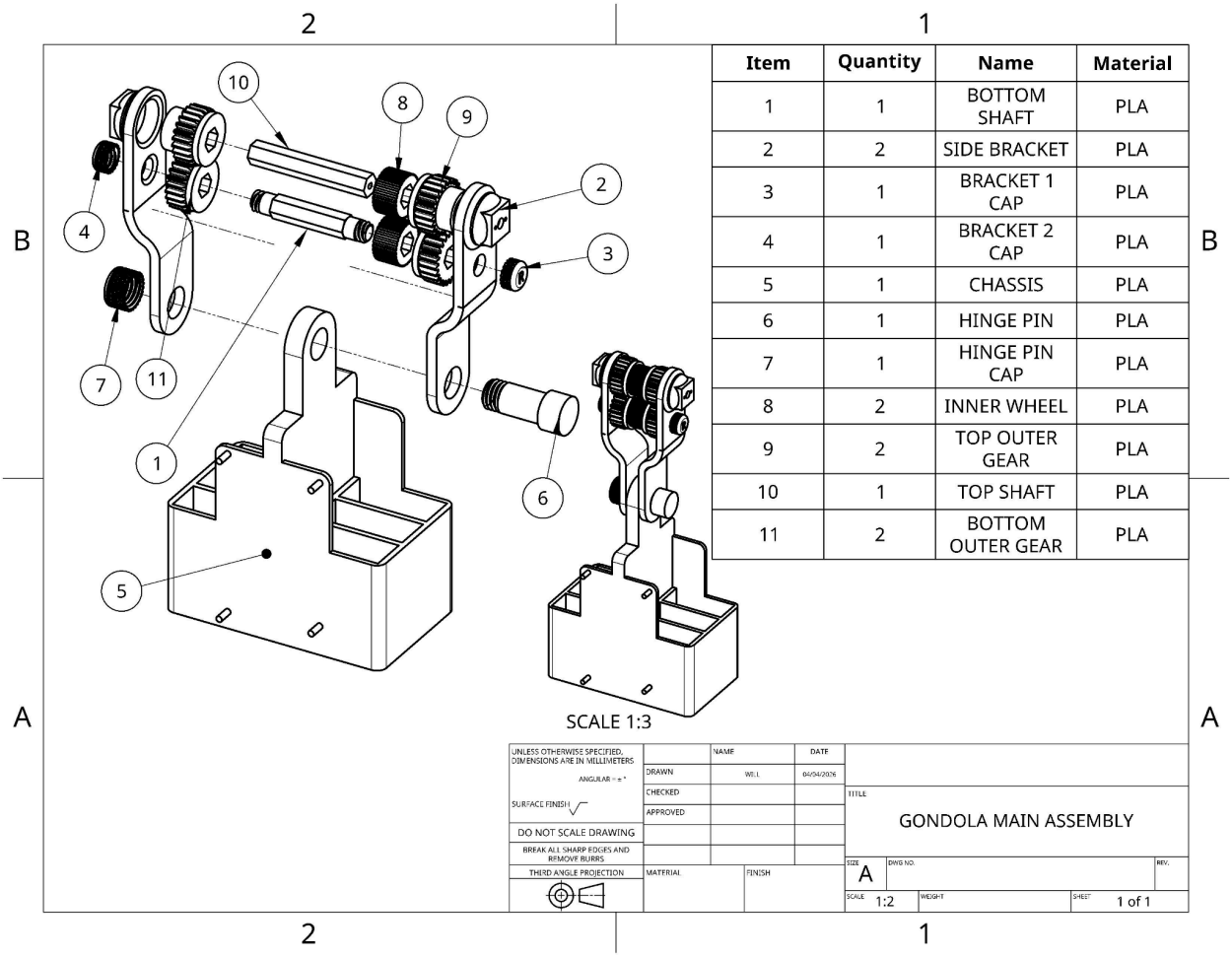
[8] "Shock absorber damper, dampers 50cc-125cc motorcycle," Amazon.ca. [Online]. Available: [https://www.amazon.ca/Absorber-Damper-Dampers-50cc%E2%80%91125cc-Motorcycle/dp/B

08J3KQ8QH](https://www.amazon.ca/Absorber-Damper-Dampers-50cc%E2%80%91125cc-Motorcycle/dp/B08J3KQ8QH). [Accessed: Apr. 12, 2026].

[9] "Arduino Due," Arduino Official Store. [Online]. Available: https://store.arduino.cc/products/arduino-due. [Accessed: Apr. 12, 2026].

Appendix A: Mechanical Drawings & Renderings

Assembly Drawing



Item	Quantity	Name	Material
1	1	BOTTOM SHAFT	PLA
2	2	SIDE BRACKET	PLA
3	1	BRACKET 1 CAP	PLA
4	1	BRACKET 2 CAP	PLA
5	1	CHASSIS	PLA
6	1	HINGE PIN	PLA
7	1	HINGE PIN CAP	PLA
8	2	INNER WHEEL	PLA
9	2	TOP OUTER GEAR	PLA
10	1	TOP SHAFT	PLA
11	2	BOTTOM OUTER GEAR	PLA

Figure 37: Assembly Drawing for Gondola

Detail Drawings

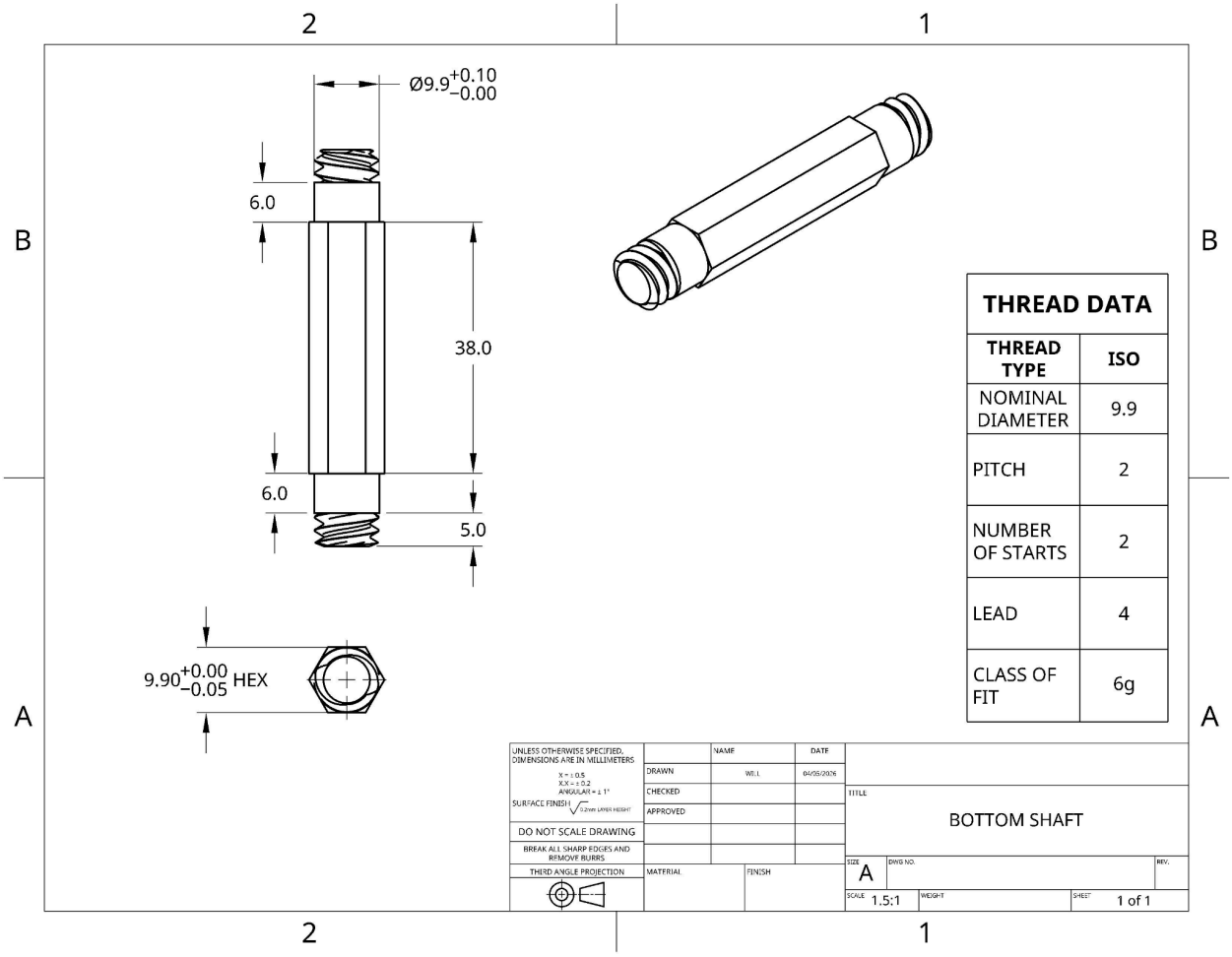


Figure 38: Detail Drawing of Bottom Shaft

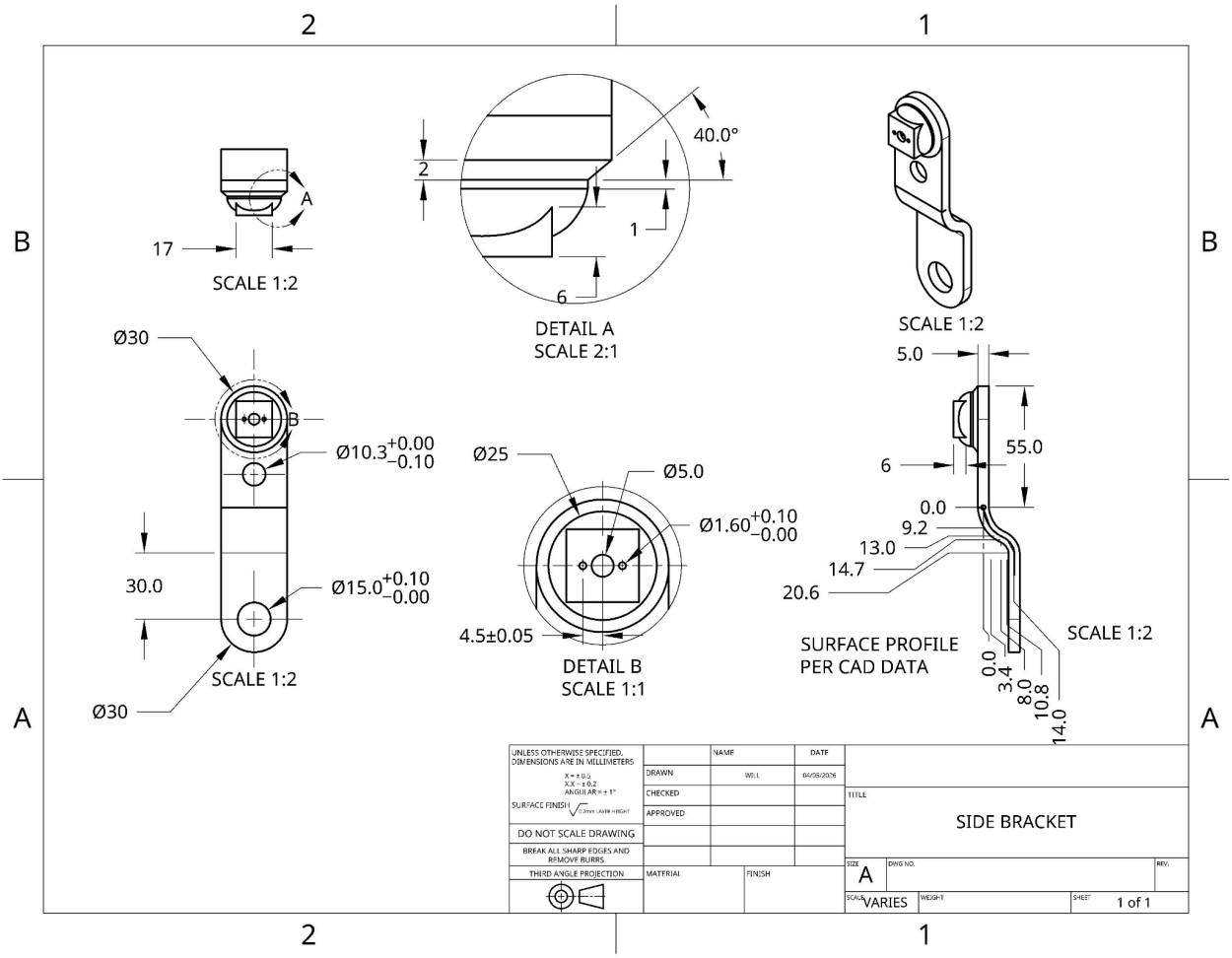


Figure 39: Detail Drawing of Side Bracket

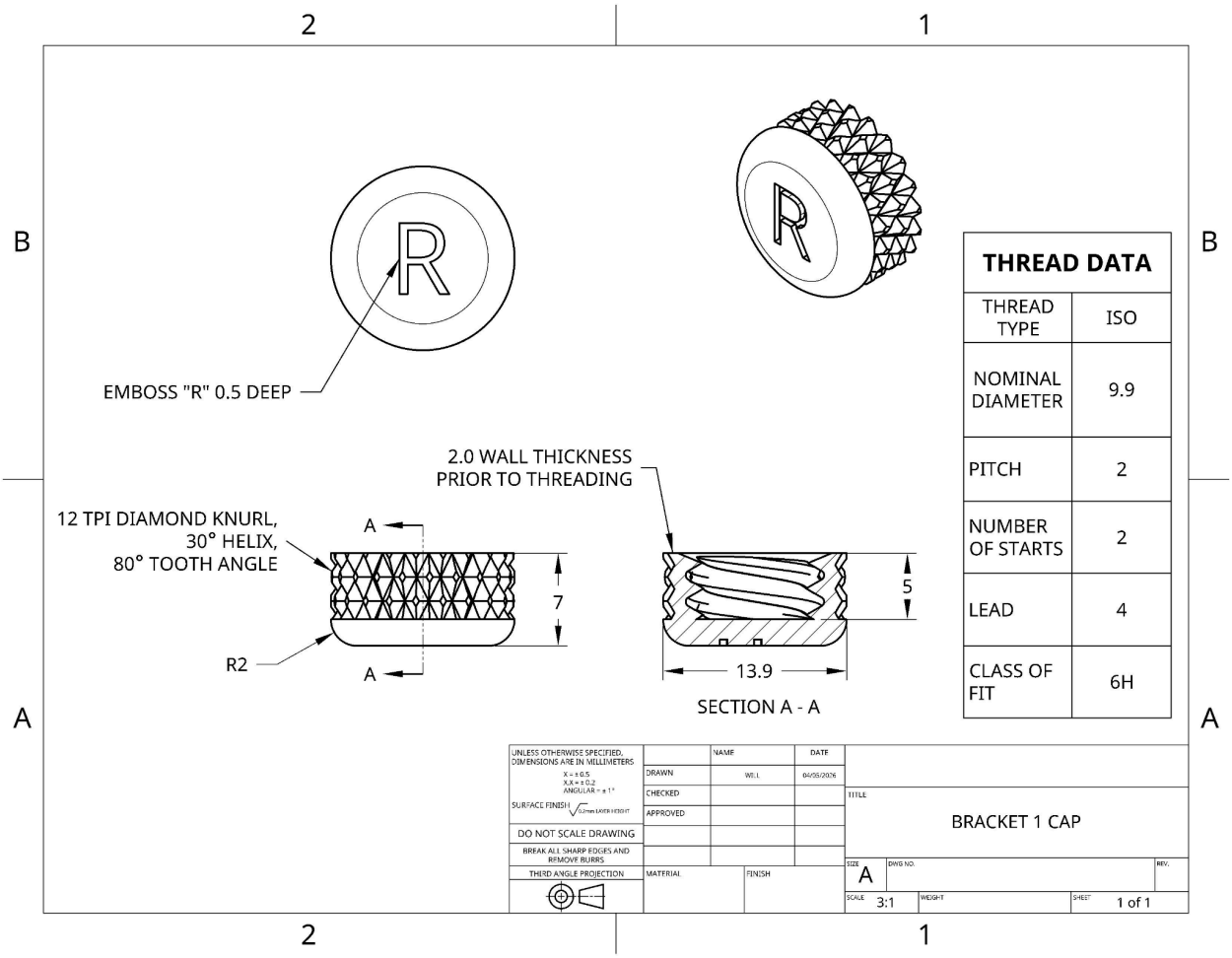


Figure 40: Detail Drawing of Bracket 1 Cap

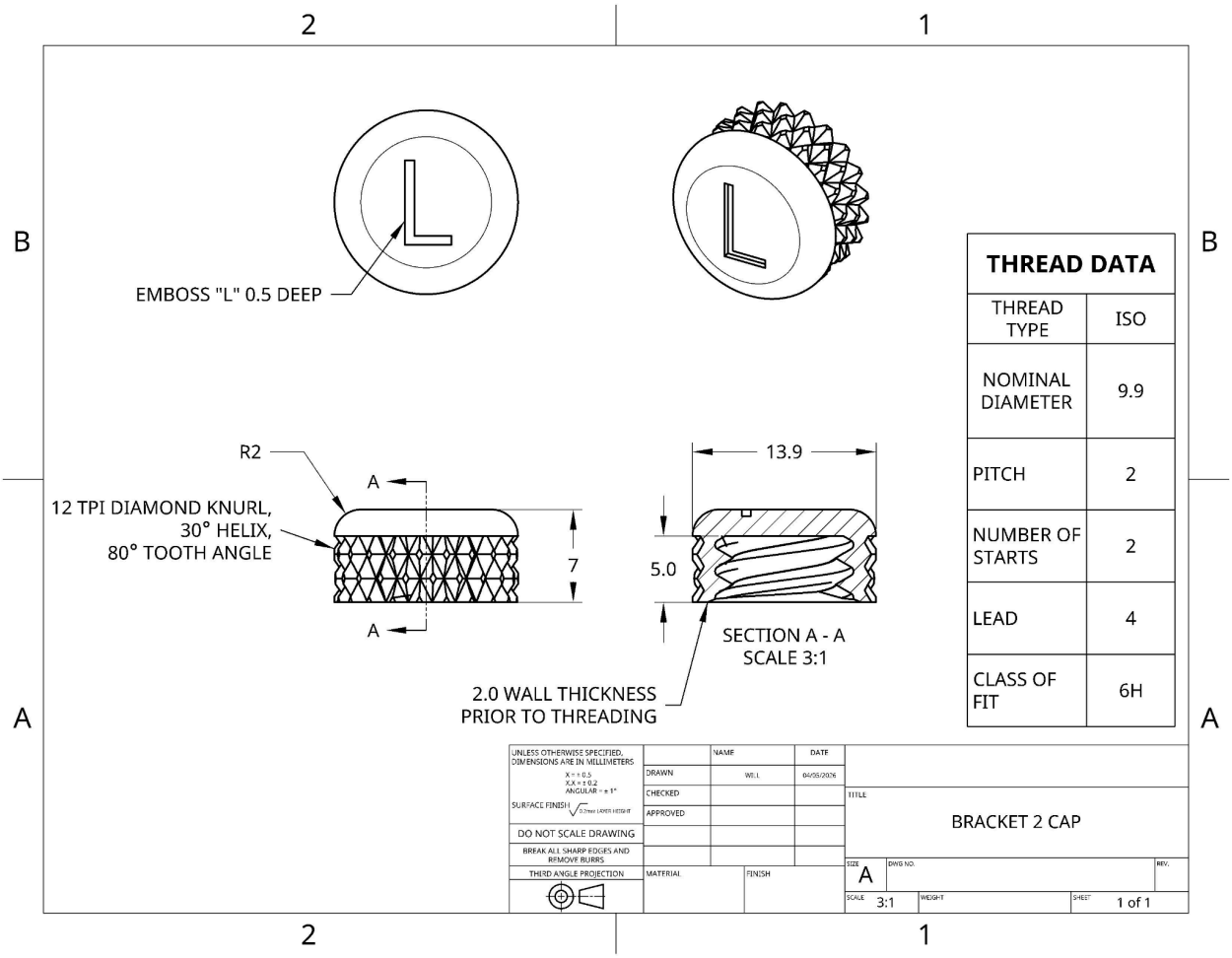


Figure 41: Detail Drawing of Bracket 2 Cap

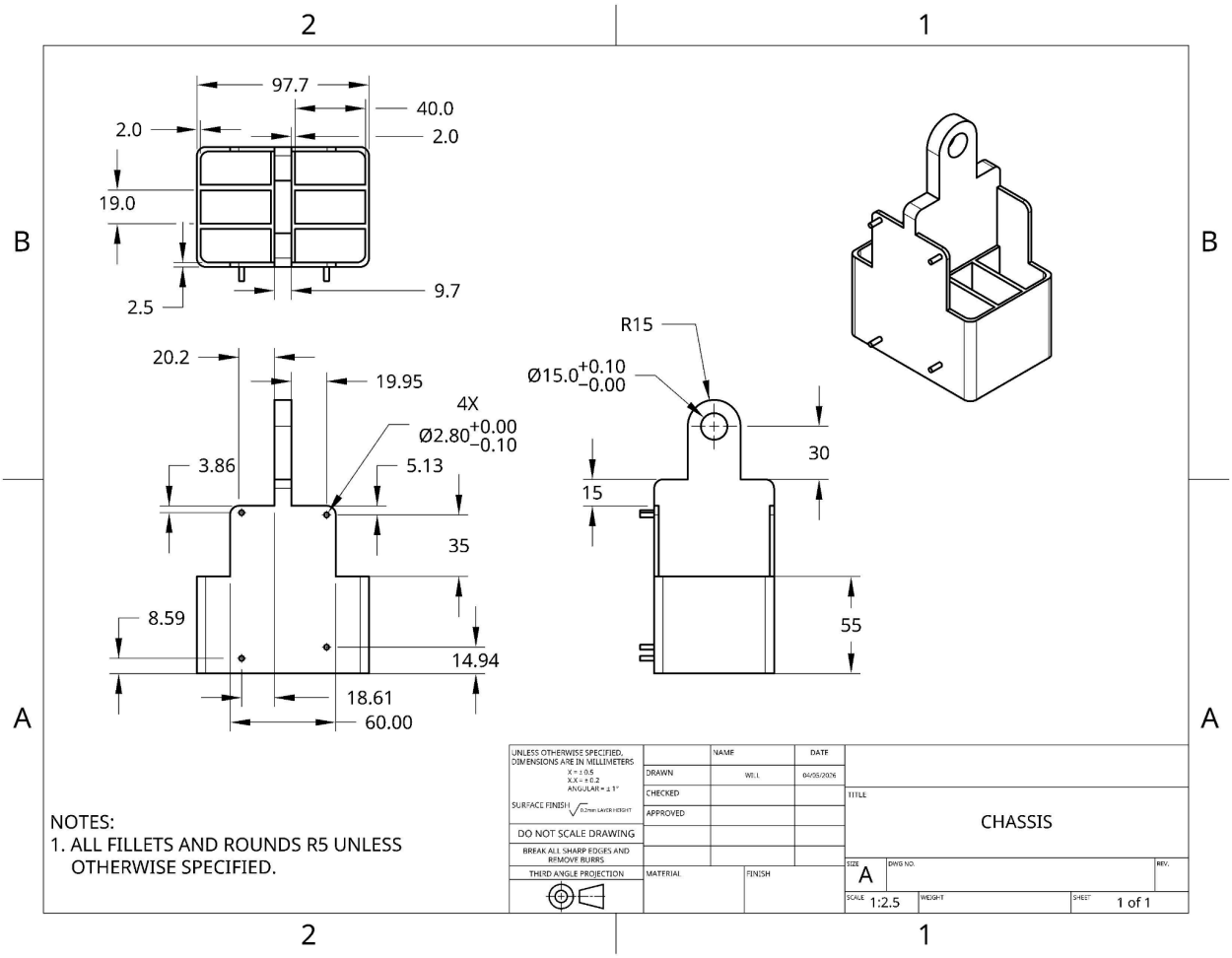


Figure 42: Detail Drawing of Chassis

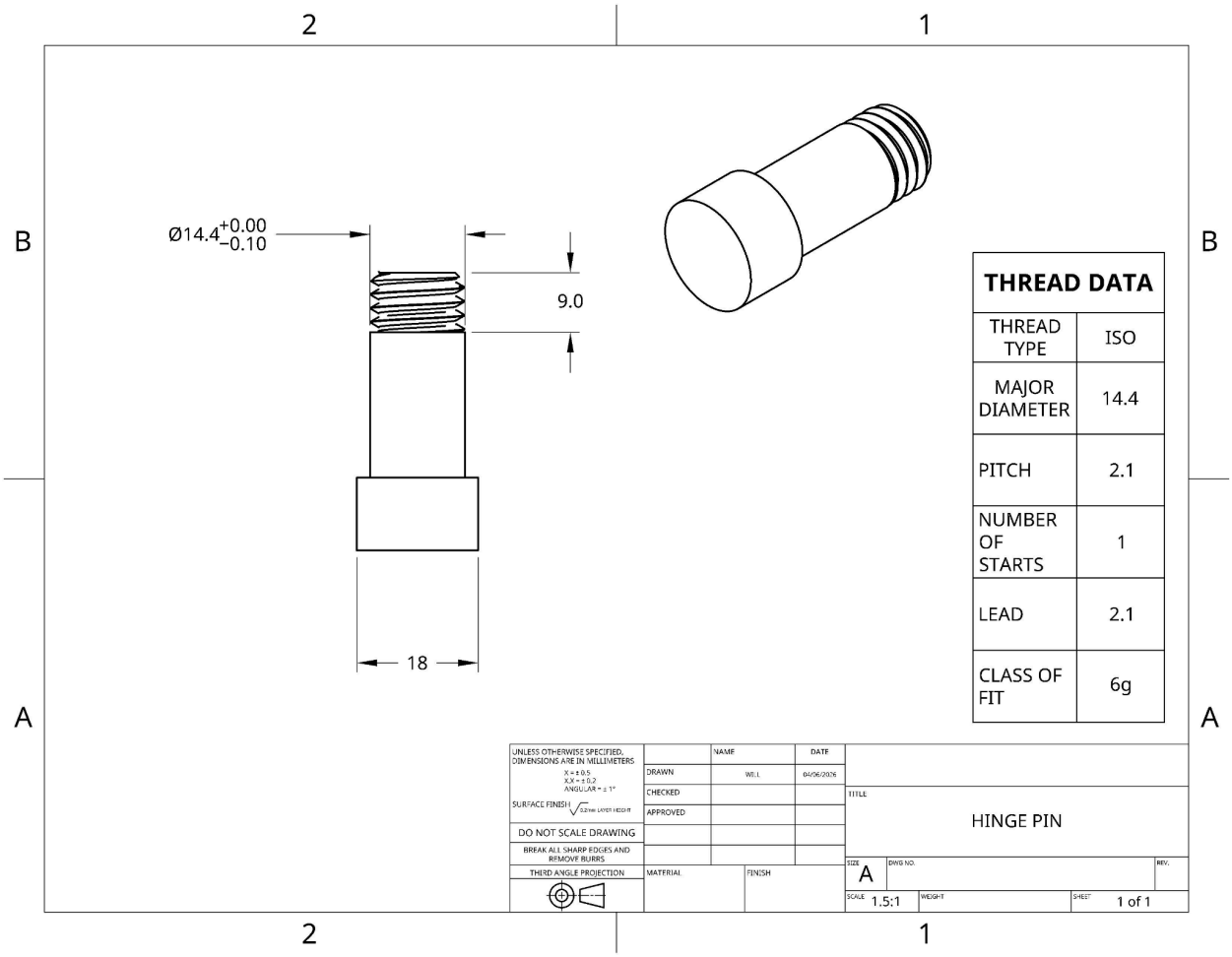


Figure 43: Detail Drawing of Hinge Pin

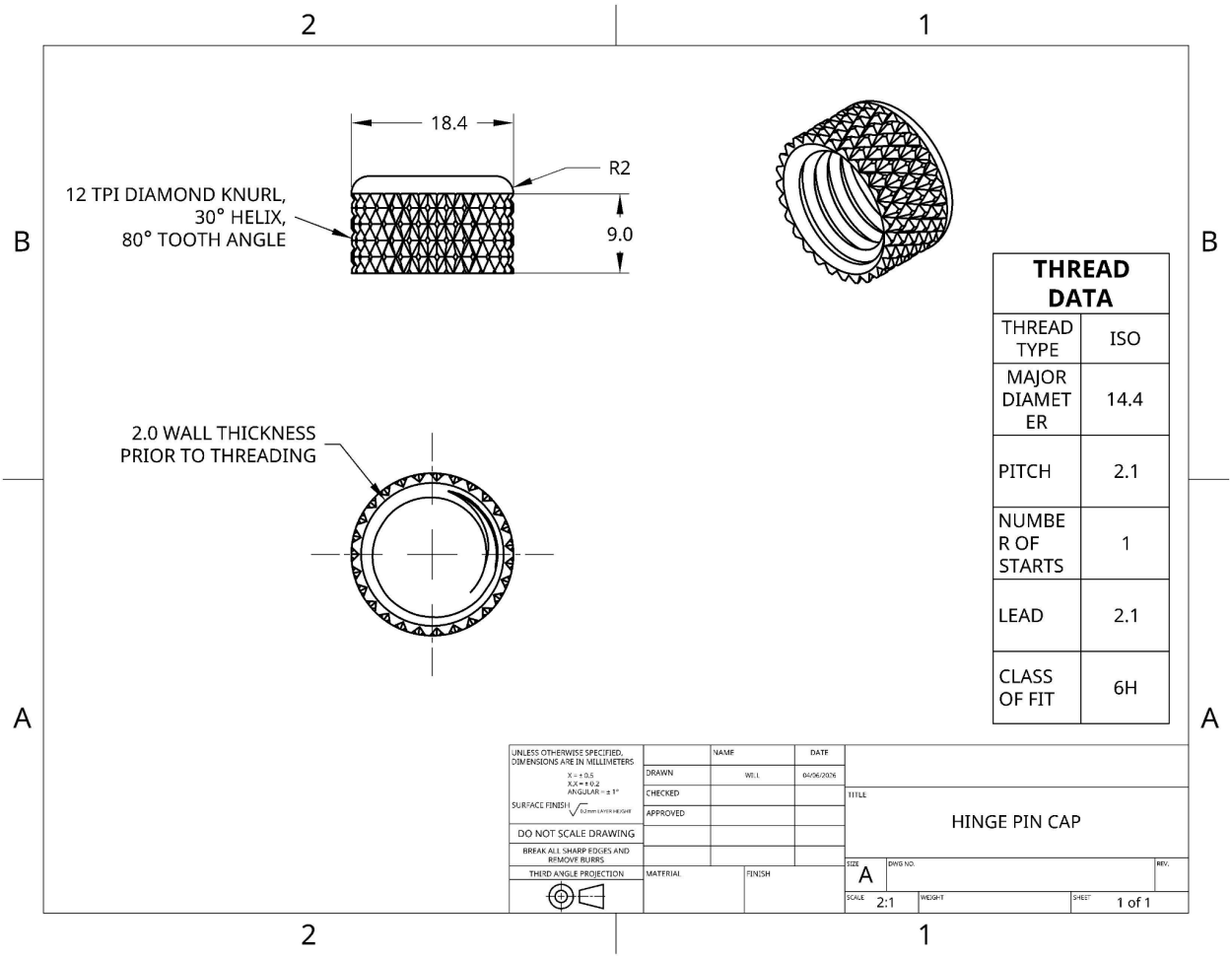


Figure 44: Detail Drawing of Hinge Pin Cap

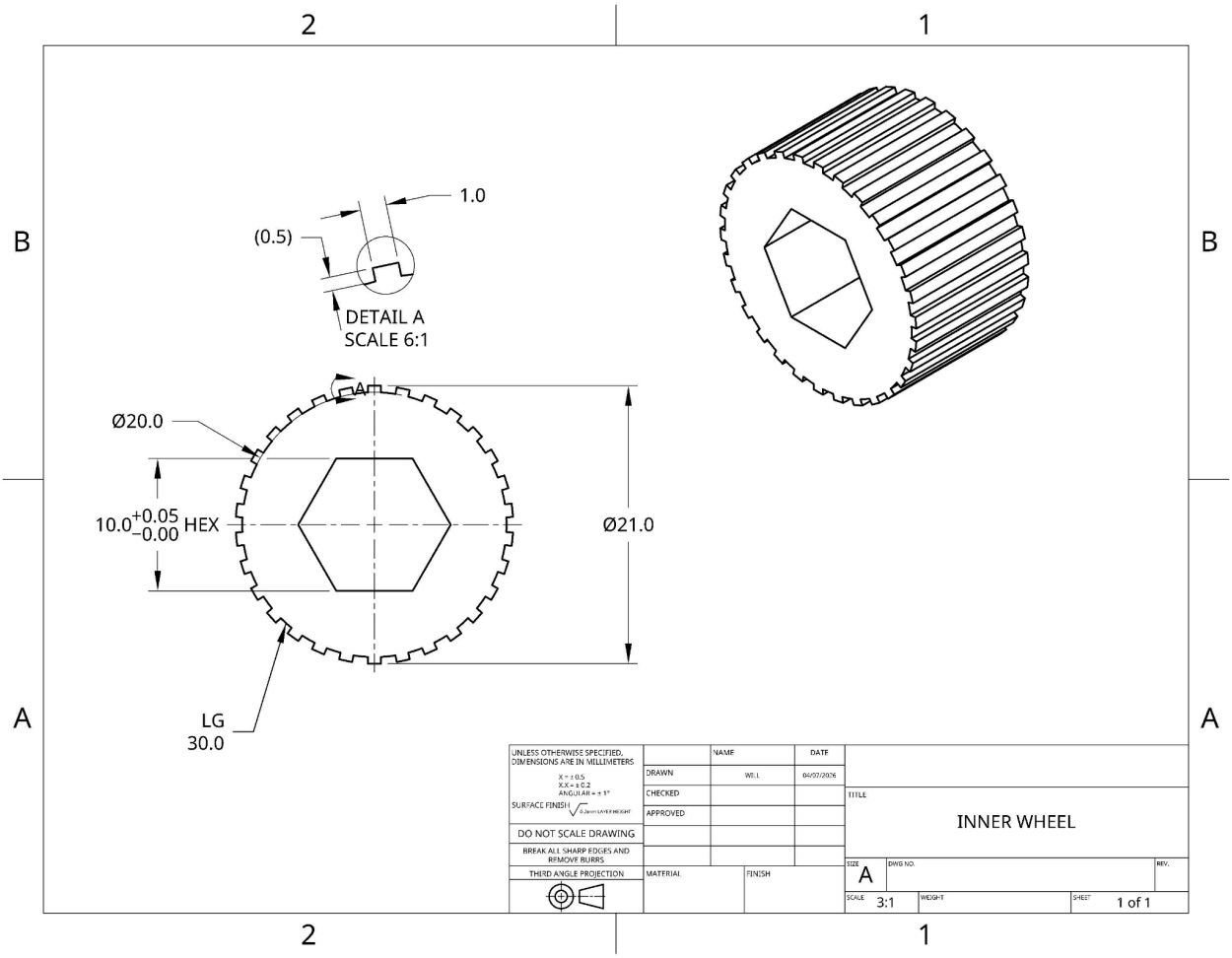


Figure 45: Detail Drawing of Inner Wheel

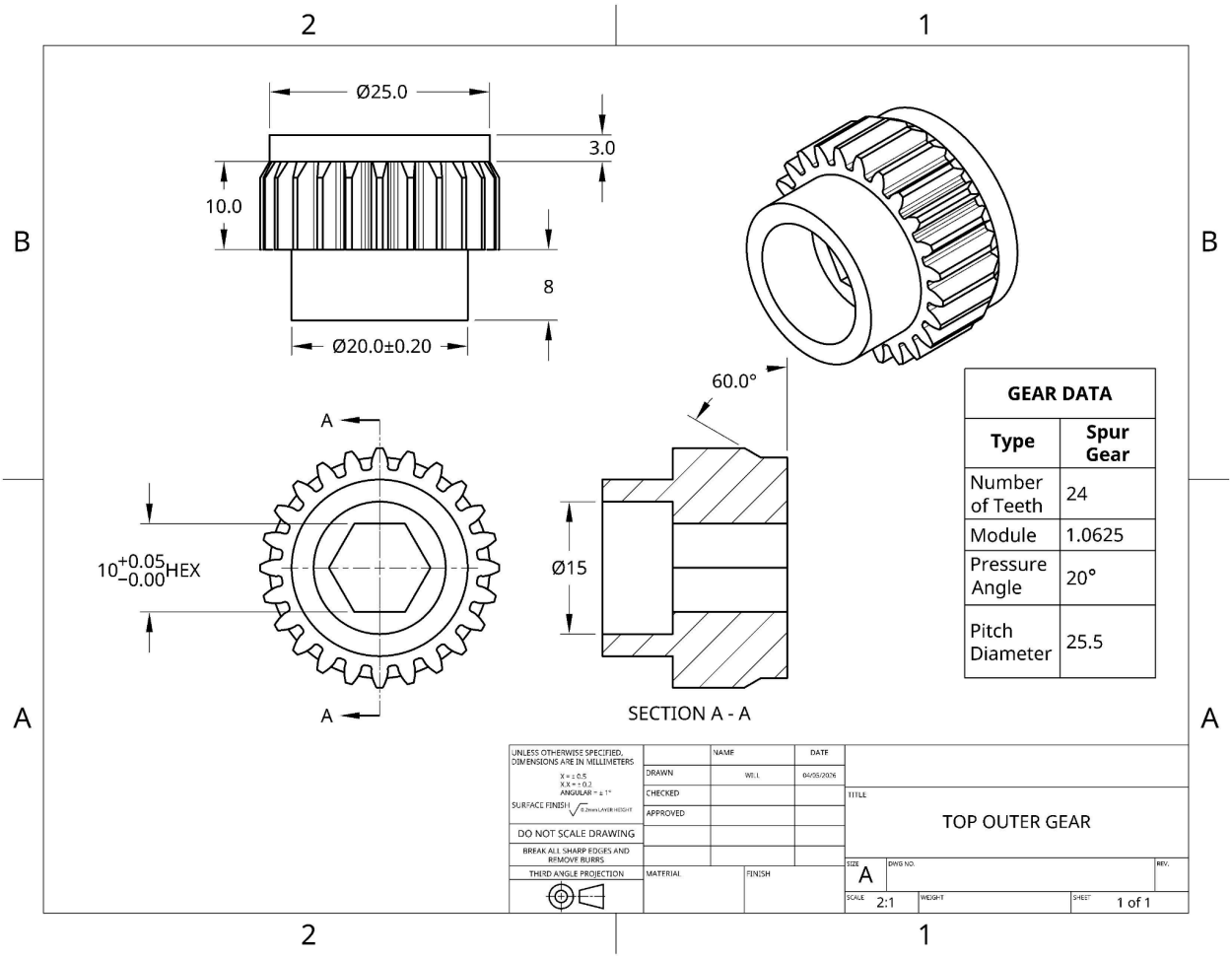


Figure 46: Detail Drawing of Top Outer Gear

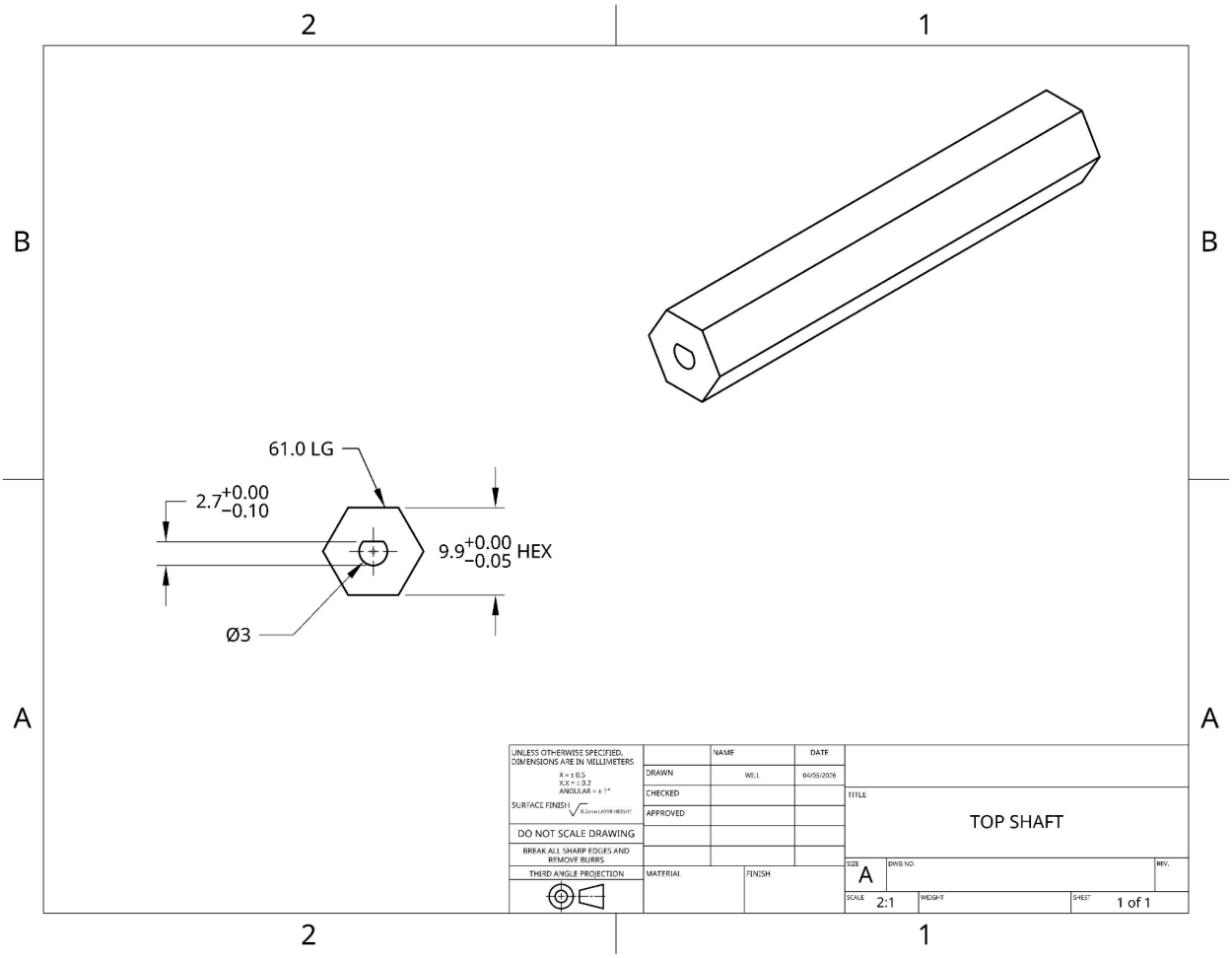


Figure 47: Detail Drawing of Top Shaft

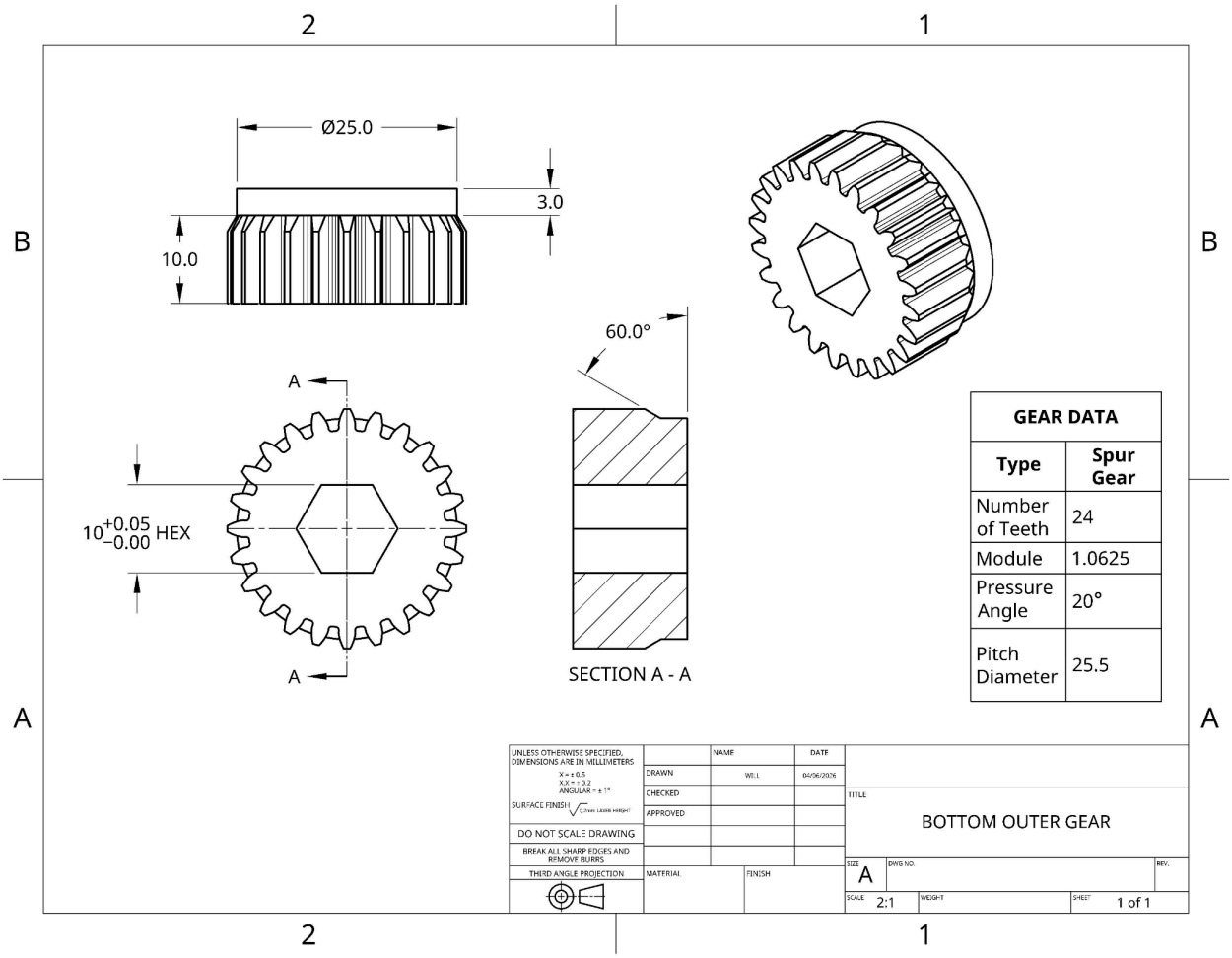


Figure 48: Detail Drawing of Bottom Outer Gear

Realistic Rendering

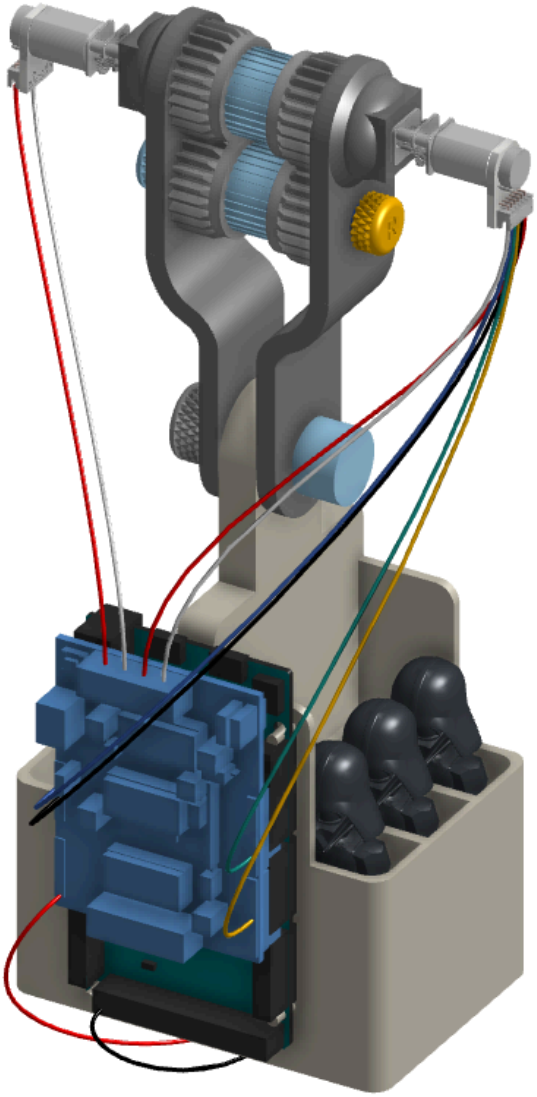


Figure 49: Realistic Rendering of the Gondola Carriage System

Appendix B: Arduino Raw Code

```
//Program for mechatronics gondola arduino board


// - - - - - - - - - - - - - - - - - - - - - - - - - -//
// ============= MODIFIABLE PARAMETERS =============//
// - - - - - - - - - - - - - - - - - - - - - - - - - -//
//These parameters are to modify for testing purposes and otherwise:

const float Default_Speed    = 250;                // Default motor speed,
from 0 to 255
const int Default_Direction = HIGH;                // Default motor
direction, LOW or HIGH

const int TimetoStop         = 600;               // How long it takes the
motor to come to a stop. This is factored in for StopTime

//const int StopTime1          = 7000;               // Time of first stop
//const int StopTime2          = 12000;              // Time of second stop
//const int StopTime3          = 15000;              // Time of third stop
//const int StopTime4          = 18000;              // Time of fourth stop

//const int WaitTime           = 5000;               // How long for
Gondola to wait after stopping before moving again
const int IncrementQuantity = 20;                  // How many times the
motor is slowed before finally stopping. This does not affect the time
taken to stop




// - - - - - - - - - - - - - - - - - - - - - - - - - -//
// ================================================//
```

```
// - - - - - - - - - - - - - - - - - - - - - - - - - -//



//Other Program Variables
float speed         = 0;    //Motor speed
int direction       = LOW;  //Motor direction

//int servo_position  = 0;    //reads servo motor position
int timer           = 0;    //timer (ms), counts ms duration of every
delay(); statement

const int sensorPin = A0;   //Sensor's pin
float sensorValue   = 0 ;   //Sensor's read value
int phase = 0;              //Current location-based phase of motor

//Pins
#define ENCA 10             // Green  Wire  (Encoder A)
#define ENCB  4             // Yellow Wire  (Encoder B)
#define EN1   3             // Motor1 Speed
#define DIR1 12             // Motor1 Direction
#define EN2  11             // Motor2 Speed
#define DIR2 13             // Motor2 Direction


volatile int position = 0;
```

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);

  //Sensor Setup
  //analogReadResolution(12);
  //analogWriteResolution(12);
  //analogWrite(DAC0, 4095);

  //Encoder Setup
  pinMode(ENCA, INPUT_PULLUP);
  pinMode(ENCB, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCA), readEncoder, RISING);

  //Motor Setup
  pinMode(EN1, OUTPUT);
  pinMode(DIR1, OUTPUT);
  pinMode(EN2, OUTPUT);
  pinMode(DIR2, OUTPUT);

}
```

```
void loop() {
  // put your main code here, to run repeatedly:

  delay(5000);
```

```
  while (true){

    //================================= Sensor
================================\\

//-------------------------------------------------------------------------
\\

    //Read sensor value from sensor pin
    sensorValue = analogRead(sensorPin);
    sensorValue = sensorValue*(3.3/4095);


    //  ==  ==  ==  ==  DEBUG PRINT STATEMENTS  ==  ==  ==  ==  //
    // ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~ - ~  //

    //Print time and motor position and motor speed
    Serial.print("Time(s): ");
    Serial.print(timer/1000);
    Serial.print("  --  --  --  --  ");

    Serial.print("Motor Position: ");
    Serial.print(position);
    Serial.print("  --  --  --  --  ");

    Serial.print("Motor Speed: ");
    Serial.print(speed);
    Serial.print("  --  --  --  --  ");

    Serial.print("Sensor Value: ");
    Serial.println(sensorValue);


    //========================= Set Motor Parameters
=========================\\
```

```
//------------------------------------------------------------------------
\\


    //switch - case statement
    //Change motor activity depending on timer value
    if (position > 40500 && phase == 2 && direction == HIGH){
      phase = 1;
      SpeedChange(direction, 0);  //Slow motor to a stop
      direction = LOW;
      delay(7000);
      SpeedChange(direction, HIGH); //Start motor, Power = HIGH (on)
      //delay(500);
      Serial.println("TEST LINE2");
      Serial.println(int(position));
      timer = timer + 7000;
    }
    //timer > 5000 ensures it wont activate right away when motor hasns't
moved yet
    else if (int(position) < 3750 && timer > 5000 && direction == LOW){
      phase = 0;
      SpeedChange(direction, 0);  //Slow motor to a stop
      direction = HIGH;
      delay(7000);
      SpeedChange(direction, HIGH); //Start motor, Power = HIGH (on)
      //delay(1000);
      timer = timer + 7000;
    }


    else if (int(position) > 37500 * 2.5/6 && int(position) < 3.25/6*37500
&& phase == 0 || int(position) > 37500 * 5.3/6 && int(position) < 37500 *
6.0/6 && phase == 1){
      if (direction == HIGH){
        phase = phase + 1;
      }
```

```
      else{
        phase = phase - 1;
      }
      if (phase < 0){
        phase = -1;
      }
      SpeedChange(direction, 0);  //Slow motor to a stop
      delay(3000);
      SpeedChange(direction, HIGH); //Start motor, Power = HIGH (on)
      timer = timer + 3000;
    }


    switch (timer){

      case 0:                          //Motor starts from full stop
        direction = Default_Direction;  //Assign direction
        SpeedChange(direction, 1); //Accelerate to default speed
        motorControls(Default_Speed, direction);//For case of error
manually set speed to max
        speed = Default_Speed;         //Update for debug window
        Serial.println("Line2");
        delay(3000);
        break;
    }




//-------------------------------------------------------------------------
\\

//=========================================================================
\\

    //Update timer to match program run time
```

```
    timer = timer + 100;
    delay(100);
  }
}
```

```
// Encoder Reading Function
//
================================================================================
======================== //
// This reads the motors position
void readEncoder(){
  int b = digitalRead(ENCB);
  if(b > 0){
    position++;
  }
  else{
    position--;
  }

}
//
================================================================================
======================== //
```

```
// MotorController
//
================================================================================
====================== //
// This apply motor variables to motor
void motorControls(int speed, int direction){
  analogWrite(EN1, speed);
  digitalWrite(DIR1, direction);
  analogWrite(EN2, speed);
  digitalWrite(DIR2, abs(direction-1));
}
//
================================================================================
====================== //
```

```
// Speed Changing Function
//
================================================================================
====================== //
// Change speed gradually for user comfort
// Power is LOW (off) or HIGH (on)
void SpeedChange(int direction, bool Power){

  //Print message to debug indicating speed change
  Serial.println("\n\n\n  --  --  --  --  --  --  --  --  --  --  --  --
--  --  --\n\n\n");
  Serial.println("Changing Motor Speed: ");
```

```
  Serial.println("\n\n\n  --  --  --  --  --  --  --  --  --  --  --  --
--  --  --\n\n\n");
  Serial.print("Power: ");
  Serial.println(Power);


  float Speed_Change;    //Amount speed changes by on each loop. Positive
if speed is increasing, negative if decreasing

  if (!Power){                                          //If motor is
stopping
    speed = Default_Speed;                              //This value will
decrease from default speed to 0
    Speed_Change = (Default_Speed-100)/IncrementQuantity*-1;  //Decrease
speed by amount proportionate to increment quantity
  }

  else{                                                 //If motor is
starting
    speed = 100;                                        //Value will
increase from 0 to default speed
    Speed_Change = (Default_Speed-100)/IncrementQuantity;   //Increase
speed by amount proportionate to increment quantity
  }

  //Changes speed for the motor a predetermined amount of times
(IncrementQuantity)
  for (int t = 0; t < IncrementQuantity; t++){

    //Print current motor speed and speed change for debug
    Serial.print("Motor Speed: ");
    Serial.print(speed);
    Serial.print(" - - - - Speed Change Quantity:");
    Serial.println(Speed_Change);
```

```
    speed = speed + Speed_Change;
    motorControls(speed, direction);          //Set motor speed
    delay(TimetoStop/IncrementQuantity);      //Wait for amt of time
appropriate for increment quantity
  }

  if (speed == 100){
    speed = 0;
    motorControls(speed, direction);          //Set motor speed
  }
  //Update timer
  timer = timer + TimetoStop;

  //Print message to debug indicating speed change over
  Serial.println("\n\n\n  --  --  --  --  --  --  --  --  --  --  --  --
--  --  --\n\n\n");
  Serial.println("Motor Speed Change Successful");
  Serial.println("\n\n\n  --  --  --  --  --  --  --  --  --  --  --  --
--  --  --\n\n\n");
}
//
=============================================================================
====================== //
```